Tracing AI Assistance and AI Agents in Survey Research*

Valentina González-Rostani† Shir Raviv‡

April 11, 2026

Abstract

Generative AI creates a measurement problem for survey research. When respondents can outsource summarization, explanation, or fact retrieval, open-ended answers may no longer reflect their own reasoning. We introduce an auditable toolkit that combines response-process paradata with prompt-specific semantic benchmarks to detect AI use in open-ended responses. We validate the approach in a randomized survey experiment in which participants complete a summary task under either blocked AI access or observed access to an embedded AI tool, and by administering the same instrument to a synthetic AI agent. AI-assisted and automated responses leave distinct behavioral and textual traces: they show reduced drafting and revision, greater similarity to AI benchmark output, and lower distinctiveness relative to peer responses. Classifiers built on these signals achieve strong out-of-sample performance. These findings show that, in the age of generative AI, authorship must be measured rather than assumed.

Keywords: survey methodology; AI; survey authenticity; paradata; embeddings

*We thank the Center for International Studies at University of Southern California and Princeton University for their financial support.

†University of Southern California, gonzalez.rostani@usc.edu.

‡Tel Aviv University.

1 Introduction

Online surveys remain central to empirical research in the social sciences because they allow researchers to recruit large and diverse samples, field complex experiments at relatively low cost, and measure attitudes and preferences at scale (Heath, Fisher, and Smith 2005; Cohen 2020; Thomas 2024). These strengths, however, depend on a basic assumption: recorded answers reflect respondents' own attention, comprehension, and judgment. The growing availability of generative AI calls that premise into question. Large language models can summarize text, generate explanations, draft open-ended answers, and even complete entire surveys on a respondent's behalf. As a result, a response may look attentive, articulate, and compliant while reflecting far less of the respondent's own reasoning than the researcher intends to measure.

This problem is not merely hypothetical. Roughly one third of crowdworkers report using LLMs on open-ended survey items (Zhang, Xu, and Alvero 2025). Recent work estimates that likely AI-assisted responding in live surveys ranges from 4.4% to 45% (Westwood and Frederick 2026; Panizza, Kyrychenko, and Roozenbeek 2026), and autonomous AI agents now pass standard survey quality checks at high rates (Westwood 2025).

Existing safeguards are poorly suited to this problem. Attention checks, CAPTCHAs, and speed flags were built to detect inattentive respondents and simple bots, not respondents who selectively outsource parts of the task to AI or automated agents that can mimic normal survey behavior (Park et al. 2025; Höhne et al. 2025; Westwood 2025). Off-the-shelf AI-text detectors offer limited help. Their performance is unstable across prompts, topics, and model versions, and they say little about how a suspicious response was actually produced (Crossley et al. 2024; Yan et al., n.d.; Claassen et al. 2026). Once AI systems can produce fluent, context-specific text that is difficult to distinguish from human writing (Dathathri et al. 2024), researchers can no longer treat apparent response quality as evidence of human authorship.

This paper addresses that measurement problem by introducing an auditable toolkit, embedded directly in surveys, to trace AI use in open-ended responses. Rather than relying on a generic detector applied after fieldwork, we instrument the response process itself. The framework combines behavioral evidence about how respondents produce their answers, including keystrokes, pauses, deletions, revisions, and task timing, with linguistic evidence from the submitted text, including markers of AI similarity, quality, originality, and semantic content. Open-ended items are especially useful for this purpose because they preserve traces of drafting and revision while also producing text that can be further use.

To validate the framework, we first conduct a randomized survey experiment. Participants read a short article on a randomly assigned topic and then write a brief summary. We randomly assign whether they complete the task without AI access or with access to a dedicated AI app built for the study, which allows us to observe use of the study-provided tool directly rather than infer AI assistance from the submitted text alone. We then administer the same instrument to a synthetic browser-based AI agent to benchmark fully automated responding. This design allows us to compare unaided human responses, human responses produced with observed AI assistance, and fully automated responses within a common task environment.

Our findings show that AI assistance changes both the content respondents submit and how they produce it. Relative to the blocked-AI control group, AI-assisted respondents submit longer and cleaner answers, but those answers show much thinner traces of drafting and revision: paste activity increases, keystroke-to-character ratios and deletion rates fall, responses move closer to prompt-specific AI benchmarks, and responses become less distinctive relative to peers. Fully automated agent responses intensify the same pattern, showing the weakest production traces, the greatest similarity to benchmark AI output, and the lowest peer originality. When we combine these behavioral and semantic measures in machine-learning classifiers, out-of-sample performance is high. Even without considering behavioral indicators, which may become less useful as agents learn to better

mimic human response behavior, AI use remains detectable through text embeddings that capture similarity to generic benchmark AI output and creativity relative to peer responses.

This paper makes two contributions. Empirically, it shows that human, AI-assisted, and fully automated survey responses leave systematically different behavioral and textual traces, even when the final answers appear fluent and on topic. Methodologically, it introduces a transparent workflow for online surveys that combines front-end instrumentation, response-process paradata, prompt-specific semantic benchmarks, and classification models, helping researchers identify prompts that are especially easy to outsource.

2 A Toolkit for Tracing AI-Assisted Responses

How should researchers study and address AI use in surveys? Existing approaches are not well-suited to this problem. AI-assisted responding can take multiple forms, including direct copying, partial borrowing, scaffolded drafting, and heavily edited transcription. Most current methods rely on one of two partial strategies. One focuses on the text itself, using classifiers to distinguish human from machine-written responses based on linguistic or semantic features (Claassen et al. 2026; Kosmyna et al. 2025; Zhang, Xu, and Alvero 2025). These methods can perform well within specific prompts or corpora, but their accuracy often declines across settings, and they reveal little about how a response was produced. The other focuses on a narrow set of behavioral signals, such as paste events or unusually rapid typing (Westwood and Frederick 2026; Deane et al. 2026; Diedenhofen and Musch 2017). These indicators provide useful initial evidence but can miss more selective forms of assistance, including cases in which respondents consult AI and then retype or lightly edit the output before submitting it (Westwood and Frederick 2026). [1]

Our approach builds on these earlier efforts by combining multiple sources of evidence within a single, auditable framework. Specifically, we integrate preventive design, process

1. For an extended discussion of existing approaches, refer to Appendix A.1.

paradata, and task-specific semantic diagnostics into a toolkit that helps researchers both identify likely AI assistance transparently and diagnose survey items that are especially easy to outsource.

We organize the toolkit around four core domains: *Behavior*, *AI signature*, *Evaluation*, and *Creativity and originality*. We then extend this core in two ways. First, we incorporate *Text embeddings*, which capture broader semantic properties of the response itself. Second, in tasks where respondents write about a shared source text, we add *Text alignment*, which measures how closely the response tracks the assigned material. The aim is not to produce a universal black-box verdict, but to preserve a transparent set of indicators that allows researchers to explain why a response appears authentic, suspicious, or difficult to classify.Table 1 summarizes these families of measures and their role in the workflow.[2]

Open-ended questions are central to this framework because they preserve evidence in both process and product. They let researchers observe how responses are written, revised, and transferred, while also evaluating the final text itself. This makes it possible to assess how a response was produced, how closely it matches AI-generated benchmarks, how distinctive it is relative to peers, how polished it appears, and, in shared-source tasks, how closely it stays anchored to the assigned text.

Behavior. This category captures how the response was produced. It includes direct indicators of transfer, such as paste events, along with broader measures of writing behavior, including task time, seconds per word, keystrokes, extra keystrokes, deletions, and the keystroke-to-character ratio. No single indicator is sufficient on its own. Taken together, however, these measures help distinguish responses drafted in the survey box through writing and revision from responses that appear to have been transferred or transcribed with little visible composition.

2. Appendix A.3 contains further details on the measurement pipeline.

Table 1: Toolkit measure families

Measure family	Illustrative measures	Authentication role
Behavior	Paste events, task time, seconds per word, keystrokes, extra keystrokes, deletions, keystroke-to-character ratio	Captures how the response was produced and whether it appears to have been drafted in the survey box.
Text embeddings	Summary-text embeddings, sentence embeddings	Captures broader semantic patterns in the response.
AI signature	Similarity to benchmark AI answer, title similarity to benchmark AI answer, novelty relative to AI benchmark	Measures proximity to prompt-specific AI output.
Evaluation	Quality index, clarity/coherence, accuracy, typo indicators	Captures apparent response quality as supplementary evidence.
Creativity and originality	Peer originality, peer distance, creativity index, lexical diversity, n-gram novelty	Captures how distinctive the response remains relative to peers.
Text alignment	Similarity to source text (if available)	Captures how closely the response tracks the assigned source text.

Text embeddings. The second category captures the overall semantic profile of the response. We convert each summary into a numerical representation of its meaning and use that representation to detect broader patterns in the text. This is useful because respondents may revise AI-generated text enough that it no longer appears copied or closely matches a benchmark answer, while still retaining broader features of AI-assisted writing.[3,4]

Evaluation. We measure the apparent quality of the final response, including clarity, accuracy, and the presence of typos, using LLM as a judge through the OpenAI API. These measures matter because AI assistance may improve the submitted answer even when the writing process shows little evidence of independent composition. We use these measures as supplementary evidence rather than standalone detectors.

3. The use of embeddings is increasing in political science. It has been used to recover latent political concepts, place actors in a common semantic space, and support inference about substantively meaningful semantic differences across groups and periods (Rheault and Cochrane 2020; Rodriguez, Spirling, and Stewart 2023; Lin 2025).

4. We encode each respondent's text using the sentence-transformer model `sentence-transformers/all-MiniLM-L6-v2`, which produces a 384-dimensional embedding. Inputs are truncated at 256 tokens, pooled using attention-mask mean pooling over token representations, and L2-normalized before entering the analysis.

AI signature. The *AI signature* category measures how closely a respondent's answer resembles benchmark AI output for the same prompt. For each item, we generate one AI response under the same instructions and compare each submitted answer to that benchmark in a shared semantic space using text embeddings (see Figure 1).

Creativity and originality. The *Creativity and originality* category captures how distinctive a response is relative to other answers to the same prompt. It includes both semantic and surface-level measures. On the semantic side, *peer originality* compares each response embedding to the centroid of peer responses, with higher values indicating greater distinctiveness. Figure 1 presents further details on this measure, and contrasts it with *AI signature*: peer originality captures distance from peers, whereas AI signature captures proximity to benchmark AI output.

We also include surface-level indicators of distinctiveness. Measures such as the type-token ratio, Yule's K, and n-gram novelty capture how varied or formulaic the writing is. These measures help detect AI assistance even when there is no direct copying or pasting, since respondents may revise or retype AI-generated text while still producing more standardized answers.

Text alignment. The final category, *Text alignment*, is task-specific by design. In our setting, respondents are asked to summarize a common source text, so it is substantively meaningful to compare each response to the article itself. Measures such as similarity to the article and faithfulness to the source capture whether the response remains anchored to the assigned text rather than drifting toward generic or externally generated content. In many open-ended survey settings, there is no common reference text, and in those cases, text alignment is not applicable.

Taken together, these categories yield an auditable profile rather than a black-box judgment. No single indicator is decisive. What matters is the pattern across multiple interpretable signals of how an open-ended response was produced and the final text.

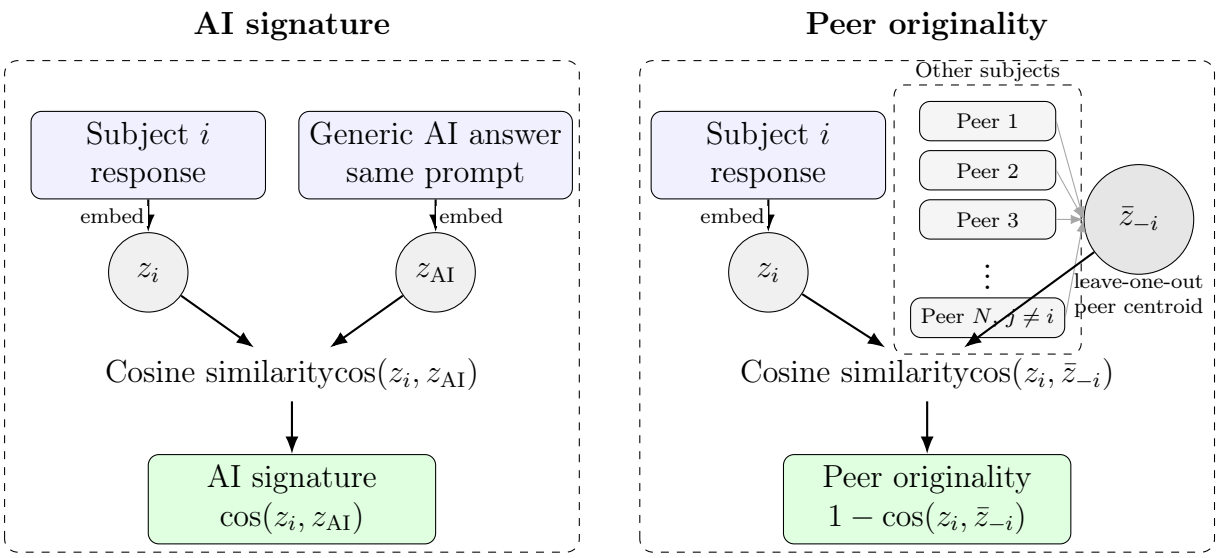


Figure 1: Embedding-based measures of AI proximity and response distinctiveness.

Notes: The left panel compares a response to benchmark AI output for the same prompt, yielding AI signature. The right panel compares the same response to the leave-one-out centroid of peer responses, yielding peer originality, defined as one minus cosine similarity. Both measures are computed in a shared semantic space and distinguish AI similarity from peer distinctiveness.

3 Validation

We validate the toolkit in a randomized survey experiment that introduces AI access under controlled conditions and then tests whether the indicators in our framework capture the resulting differences in response production. We recruited participants through CloudResearch Connect, a widely used online platform for survey experiments in political behavior (e.g., Stagnaro et al. 2024). Participants completed a short survey that included an open-ended task: they read a brief article and then wrote a short summary in their own words.

We chose this task because it closely matches common practice in political behavior research. Many survey experiments expose respondents to a short text and then use follow-up items to assess whether they attended to the information, understood it, and could restate its main message (e.g., Ziegler 2022; Kane and Barabas 2019). The value of such a task rests on a key assumption: that respondents actually processed the article and produced the response on their own. As AI tools make summarization easy to outsource, that assumption becomes less secure. A response may appear fluent and accurate while

revealing less about the respondent's own effort and comprehension. The summary task therefore provides a realistic and substantively meaningful setting for studying AI-assisted responding.

We randomly varied whether respondents were given access to AI while completing the task. In the control condition, respondents were instructed not to use external AI tools. In the treatment condition, respondents were informed that an AI writing assistant was available and that its use was entirely optional. To create a controlled setting in which AI access was observed rather than inferred post hoc, we developed our own LLM-based app and embedded it directly in the survey using Streamlit and the OpenAI API. This allowed us to track tool use and link it to individual respondents. By embedding the tool within the survey, we can more directly assess how access to AI affects both the response process and the text respondents ultimately submit.[5]

We also randomized the topic of the article to be summarized, assigning each respondent to one of seven short texts. This reduces the risk that the results are driven by idiosyncratic features of any single article and helps ensure that the patterns we identify are not specific to one topic or prompt. Appendix A.2 provides additional details on the experimental design and reproduces the full text of all task materials.

To operationalize the toolkit, we instrument the open-ended task to capture both process- and text-based outcomes. On the process side, we record paste attempts, deletion and revision behavior, keystroke activity, and response timing. On the text side, we analyze the summary using measures of similarity to benchmark AI outputs, to the source article, and to peer responses, respectively. Our main validation test examines whether experimentally introduced AI access alters how respondents produce open-ended responses. In later sections, we extend this analysis by asking whether the same indicators

5. As part of a separate preregistered project on how AI use shapes policy attitudes, the broader experiment also included two additional treatment arms that varied the framing and perceived quality of the AI tool. Because those conditions served different substantive purposes, we focus here on the cleanest comparison: respondents assigned to the blocked-AI control condition (n=412) and respondents assigned to the standard AI-access condition (n=416). Appendix A.7 reports results for the full sample (n=1,647), controlling for the additional treatment arms, and shows that the main patterns are unchanged in both direction and magnitude.

also distinguish fully automated survey completion by an AI agent.

3.1 Behavioral and Textual Effects of AI Access

Figure 2 displays the effects of AI assistance relative to the blocked-AI control group, expressed as topic-adjusted standardized contrasts. Across these outcomes, AI access is associated with longer submitted text, substantially more paste activity, fewer keystrokes relative to the amount of text retained, and shorter completion times. The magnitude of these differences is also large. For example, AI assistance reduces the keystrokes-per-character ratio by 1.040, a drop more than twice the control-group level. This indicates a strong shift away from in-box drafting and toward paste-based production, where much of the final text is inserted rather than typed. The key point is not simply that assisted responses look different at the point of submission. They are produced through a different workflow. Relative to blocked access, the behavioral trace shifts away from iterative drafting inside the survey box and toward transfer, light editing, and efficient transcription of externally generated text.

The semantic plots point in the same direction. Figure 2 shows that AI-exposed responses move closer to the study-specific AI benchmark and that peer distinctiveness also decline. Furthermore, the evaluator identifies fewer typos in AI-assisted answers relative to the control. This is precisely the pattern that creates a survey-validity problem: the final answer can remain on topic and look competent while becoming less diagnostic of the respondent's own processing.

AI access often makes responses more homogeneous, even though the language can be more diverse; they are well-drafted, longer, and reduce evidence of respondent effort. If a researcher looked only at the final text, AI-exposed respondents could appear more articulate, more accurate, or more engaged. The paradata and novelty suggest a different interpretation: some of the apparent improvement reflects changes in production conditions rather than deeper processing of the stimulus.

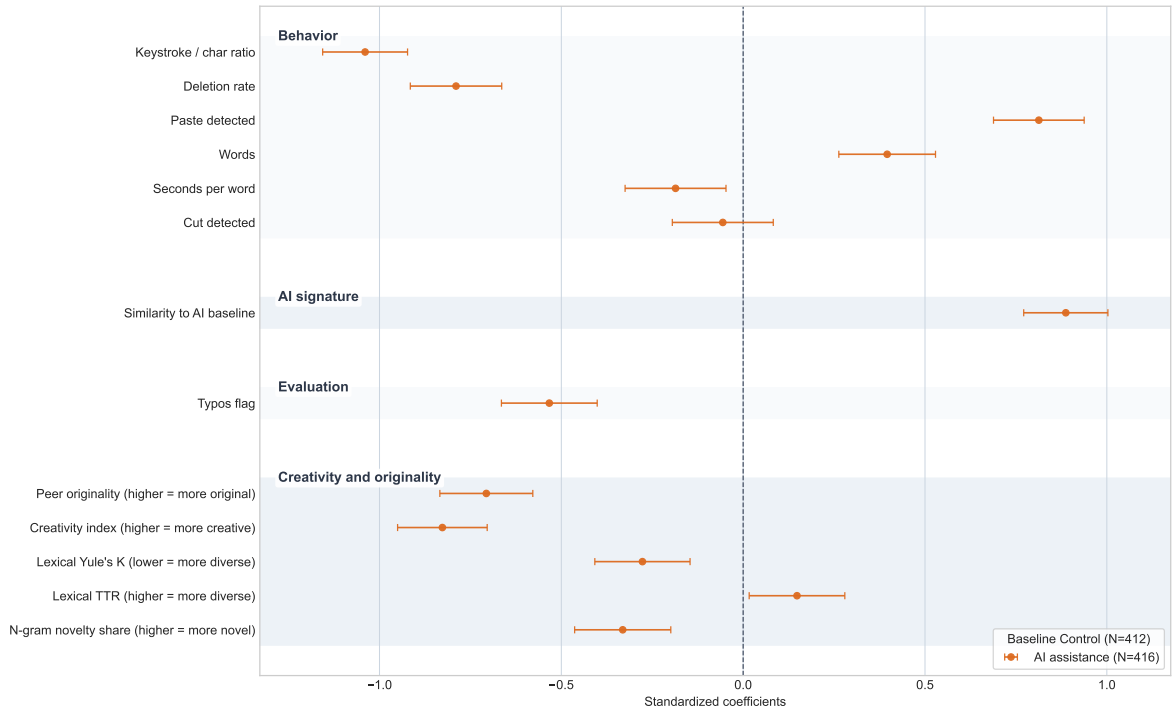


Figure 2: Standardized differences in summary-task outcomes between the human AI-only treatment and the human control group.

Notes: Each point is the coefficient on an indicator for assignment to the AI assistance arm from a separate topic-adjusted OLS regression with article-topic fixed effects; horizontal bars show 95% HC1 confidence intervals. All dependent variables are standardized, so coefficients are in standard-deviation units rather than raw units. Positive coefficients indicate higher values in `AI assisted` than in the baseline control group; negative coefficients indicate lower values. The *Behavior* category captures the writing process: *Words* is final summary length, *Seconds per word* captures production speed, *Keystroke / char ratio* and *Deletion rate* capture editing effort, and *Paste detected* and *Cut detected* are binary indicators for whether those actions occurred during writing. The *AI signature* measure is similarity to an AI baseline summary. The *Evaluation* measure, *Typos flag*, is a binary indicator coded one when evaluators flagged typos. The *Creativity and originality* outcomes capture peer-rated originality and creativity, lexical diversity (*Lexical TTR*; *Yule's K*, where lower raw values imply greater diversity), and n-gram novelty. Refer to Figure A.17 for additional measures, and refer to Figure A.16 for other AI arms, such as low-quality AI provided. Refer to the full table in Table A.1.

3.2 Validation with a Synthetic AI Agent

We also administered the same survey to a `SyntheticSurveyAgent` to benchmark fully automated responding. Prior research shows that LLMs can generate synthetic respondents (Argyle et al. 2023; Suguri Motoki et al. 2024; Sarstedt et al. 2024). We build on that work, but for a different purpose. Rather than using the agent to simulate a population, we use it to test the survey design and to establish a benchmark for what fully automated responding looks like. This allows us to examine how behavior and outputs shift across different forms of AI assistance.

The agent completed the survey through the Chrome browser channel and used

`gpt-4o-mini` to answer both closed- and open-ended items. Each completion was conditioned on a pre-specified persona profile to approximate a plausible survey participant rather than an unconstrained model producing generic text. In total, we obtained 543 completed agent responses. We compare these responses to the same human control group used as the non-AI baseline and to the human AI-assistance group. This places fully automated and AI-assisted human responses on a common scale and allows us to assess whether the toolkit captures a shared response signature rather than overfitting to a single mode of AI use.

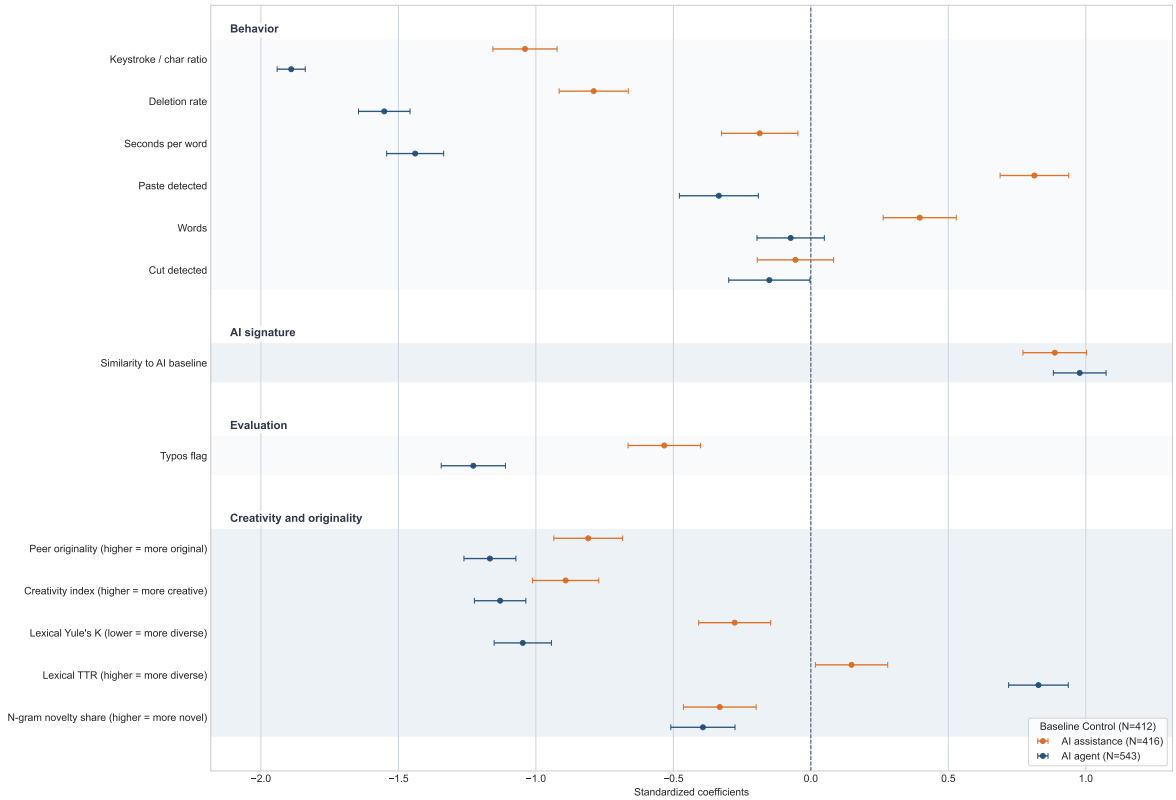


Figure 3: Standardized differences in summary-task outcomes for AI-assisted human responses and AI-agent responses, each relative to the human control group.
Notes: The figure overlays two separate contrasts against the same baseline human control group. The *AI assistance* series compares human responses in the AI-only treatment to the control group. The *AI agent* series compares agent-generated responses to the same control group. Each point is estimated from a separate topic-adjusted OLS model with article-topic fixed effects, and horizontal bars indicate 95% HC1 confidence intervals. All dependent variables are standardized within the relevant comparison sample, so coefficients can be interpreted as differences in standard-deviation units. Positive coefficients indicate higher values than the human control group within that comparison, whereas negative coefficients indicate lower values. The outcome categories are the same as in Figure 2: *Behavior* captures writing effort, speed, and editing actions; *AI signature* captures similarity to an AI baseline summary; *Evaluation* includes the typo-flag indicator; and *Creativity and originality* captures originality, creativity, lexical diversity, and novelty. Refer to the full table in Table A.2.

Figure 3 shows a consistent ordering across the most diagnostic measures. Human AI

assistance already lowers keystroke-to-character ratios, reduces deletion, increases paste incidence, and raises similarity to the AI benchmark relative to unaided human writing. The AI agent intensifies those same patterns: it leaves the thinnest drafting trace, shows the greatest benchmark similarity, and exhibits the lowest distinctiveness relative to peer responses. In many behavioral measures, the agent's effect relative to the control group is roughly twice as large as the effect of human AI assistance. By contrast, indicators based on similarity to the AI benchmark show little difference between the AI-assisted human group and the AI agent, which is consistent with the fact that both rely on AI-generated text. Measures based on peer distance and creativity, however, separate the groups more clearly, with the AI agent appearing even less distinctive relative to peer responses than AI-assisted writing.

4 Classification and Predictive Performance

Besides introducing the toolkit, we also train a model to predict AI usage. Researchers could use the toolkit on its own, but the machine learning exercise helps identify which indicators are most informative for detection. The best out-of-sample models exceed 0.9 ROC-AUC (see Figure 4). Here, the top-performing model is a random forest, which combines evaluation, AI-signature measures (similarity to generic AI output), and behavioral indicators. The strongest standalone signals are behavioral, followed by AI signature and peer originality.[6] We also implement a sensitivity test that excludes behavioral indicators and instead aims to distinguish AI agents from all human-collected data, pooling unassisted and AI-assisted human responses into a single comparison group.[7] Even under this restriction, we are able to train a high-accuracy classifier, with the best-performing model relying on AI signature, evaluation, and creativity indicators (see Figure A.22).

The appropriate practical use is not to impose a universal exclusion threshold. A bet-

6. Appendix Table A.3 reports the standalone contribution of each indicator.

7. This sensitivity analysis is important as individuals creating AI agents may become more experts on its design and could eventually mimic human behavior when writing.

ter approach is to calibrate study-specific cutoffs on transparent indicators or to generate an AI-likely score from those same measures and then report how the rule was chosen. The workflow is most defensible when the analyst can show, before seeing the substantive results, which indicators matter, why they matter, and how the rule performs in validated data. That approach fits contemporary standards for survey-quality reporting better than reliance on a proprietary detector with no clear decision logic.

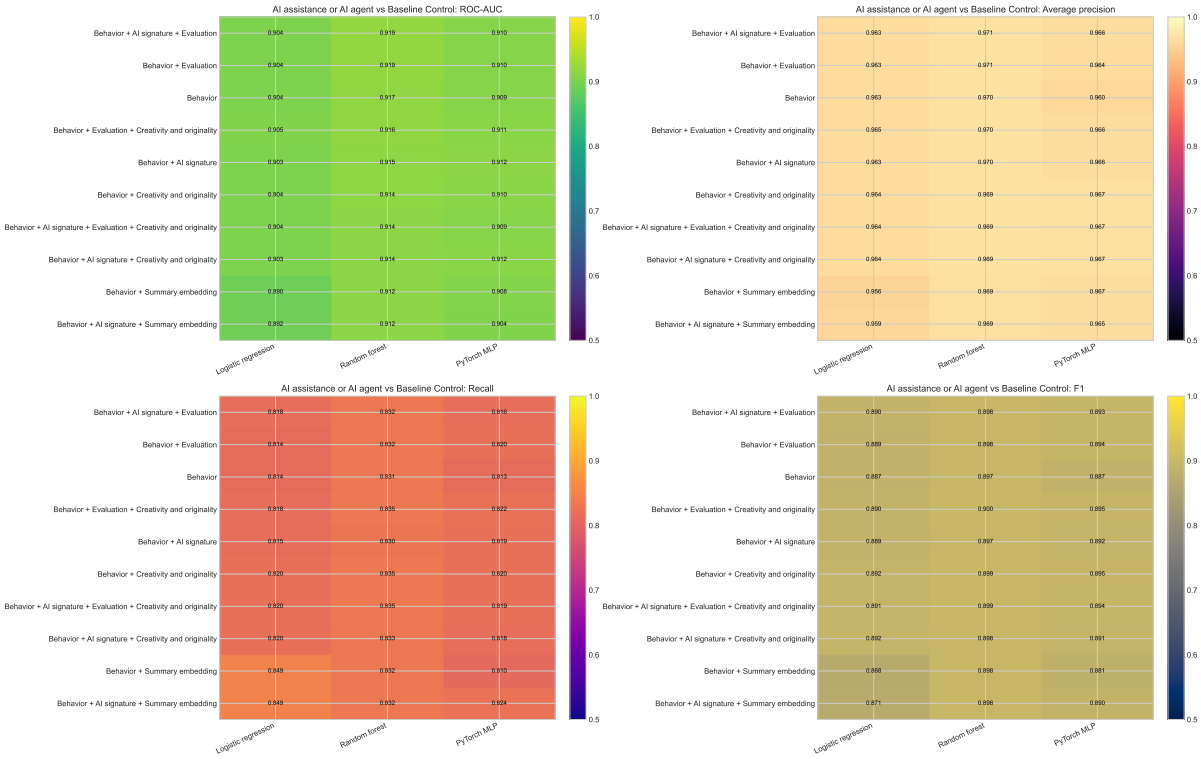


Figure 4: Top classifier combinations: AI assistance or AI agent vs. Baseline Control

Notes: Each point is the coefficient on an indicator for assignment to the AI assistance arm from a separate topic-adjusted OLS regression with article-topic fixed effects. Horizontal bars show 95% HC1 confidence intervals. All dependent variables are standardized, so coefficients are expressed in standard-deviation units rather than raw units. Positive coefficients indicate higher values in the `AI assisted` group than in the baseline control group; negative coefficients indicate lower values. The *Behavior* category captures the writing process: *Words* measures final summary length; *Seconds per word* captures production speed; *Keystroke / char ratio* and *Deletion rate* capture editing effort; and *Paste detected* and *Cut detected* are binary indicators for whether those actions occurred during writing. The *AI signature* measure captures similarity to an AI baseline summary. The *Evaluation* measure, *Typos flag*, is a binary indicator coded as one when evaluators flagged typos. The *Creativity and originality* outcomes capture peer-rated originality and creativity, lexical diversity (*Lexical TTR*; *Yule's K*, where lower raw values indicate greater diversity), and n-gram novelty. Figure A.17 presents additional measures, and Figure A.16 reports results for other AI arms, including the low-quality AI condition. The full regression table appears in Table A.1.

5 Implications for Survey Research

The evidence in this study shows that response authenticity can no longer be inferred from a polished final answer. AI-assisted and fully automated responses are often fluent, relevant, and apparently compliant, which creates a growing problem for survey research as data collection increasingly relies on online panels, crowdsourcing platforms, and other digital channels that are fast, scalable, and inexpensive. Those advantages matter only if responses reflect respondents' own thinking. Inauthentic participation is not new, but earlier forms, such as inattentive respondents or simple bots, often left more visible traces. Generative AI changes the character of the problem by producing credible-looking answers even when much of the cognitive work has been outsourced. The implications extend beyond survey research. Public opinion polls help democratic systems register and communicate citizen preferences (Westwood 2025), yet AI tools now make it easier to fabricate that signal through persuasive synthetic responses that mimic grassroots participation (Goldstein et al. 2024). As a result, distinguishing genuine public sentiment from coordinated manipulation becomes more difficult.

In this paper, we take a first step toward addressing this challenge by introducing a practical framework for identifying likely AI involvement through open-ended survey responses. Rather than relying on the final text alone, our approach combines multiple forms of evidence about how a response was produced. We find that AI-assisted and fully automated responses leave identifiable traces. AI-produced responses are often longer, cleaner, and closer to benchmark AI output, yet they also show thinner traces of drafting and revision and lower distinctiveness relative to peers.

At the same time, using open-ended items to assess respondent authenticity is not costless. Adding such questions can lengthen the questionnaire, increase completion time, raise administrative costs, and contribute to respondent fatigue, with possible downstream effects on answers to later items in the survey (Anduiza and Galais 2017). For that reason, we do not view authentication items as disposable add-ons. Wherever pos-

sible, researchers should design them to serve more than one purpose (e.g., substantive outcome or second-order beliefs).

Our findings point to a further concern: AI agents that can navigate websites and complete tasks with little human oversight. The threat is no longer hypothetical, yet fully automated participation continues to face practical frictions, including monetary costs, platform safeguards, account constraints, and other barriers that limit routine large-scale deployment. In many low-paying online survey settings, ordinary respondents may still find partial AI assistance more attractive than full automation, which requires more resources, more setup, and carries greater detection risk. These constraints will not remove the problem, and they may weaken over time. But they suggest that researchers still have some room to adapt before such tools become easy to deploy at scale.

More broadly, generative AI is a disruptive technology for research. Our framework offers a starting point for identifying likely forms of AI involvement, not a final solution. In a fast-changing environment, protecting research integrity will require researchers to remain adaptive and methodologically creative, revising their tools and assumptions as new challenges emerge.

References

Anduiza, Eva, and Carol Galais. 2017. Answering without reading: imcs and strong satisficing in online surveys. *International Journal of Public Opinion Research* 29 (3): 497–519.

Argyle, Lisa P., Ethan C. Busby, Nancy Fulda, Joshua R. Gubler, Christopher Rytting, and David Wingate. 2023. Out of One, Many: Using Language Models to Simulate Human Samples. *Political Analysis* 31 (3): 337–351.

Bell, Andrew M., and Thomas Gift. 2023. Fraud in Online Surveys: Evidence from a Nonprobability, Subpopulation Sample. *Journal of Experimental Political Science* 10 (1): 148–153.

Claassen, Joshua, Jan Karem Höhne, Ruben Bach, and Anna-Carolina Haensch. 2026. Identifying Bots Through LLM-Generated Text in Open Narrative Responses: A Proof-of-Concept Study. *Social Science Computer Review,* 08944393251408022.

Cohen, Denis. 2020. Between strategy and protest: how policy demand, political dissatisfaction and strategic incentives matter for far-right voting. *Political Science Research and Methods* 8 (4): 662–676.

Cohen, Mollie J., and Zach Warner. 2021. How to Get Better Survey Data More Efficiently. *Political Analysis* 29 (2): 121–138.

Crossley, Scott, Yu Tian, Joon Suh Choi, Langdon Holmes, and Wesley Morris. 2024. Plagiarism Detection Using Keystroke Logs, 476–483.

Dathathri, Sumanth, Abigail See, Sumedh Ghaisas, Po-Sen Huang, Rob McAdam, Johannes Welbl, Vandana Bachani, Alex Kaskasoli, Robert Stanforth, Tatiana Matejovicova, et al. 2024. Scalable watermarking for identifying large language model outputs. *Nature* 634 (8035): 818–823.

Deane, Paul, Mo Zhang, Jiangang Hao, and Chen Li. 2026. Using Keystroke Dynamics to Detect Nonoriginal Text. *Journal of Educational Measurement* 63 (1): e12431.

Diedenhofen, Birk, and Jochen Musch. 2017. PageFocus: Using paradata to detect and prevent cheating on online achievement tests. *Behavior Research Methods* 49 (4): 1444–1459.

Fuchs, Anna, Anna-Carolina Haensch, and Wiebke Weber. 2026. AI for Survey Design: Generating and Evaluating Survey Questions with Large Language Models. *SocArXiv.*

Goldstein, Josh A, Jason Chao, Shelby Grossman, Alex Stamos, and Michael Tomz. 2024. How persuasive is ai-generated propaganda? *PNAS nexus* 3 (2): pgae034.

Heath, Anthony, Stephen Fisher, and Shawna Smith. 2005. The globalization of public opinion research. *Annu. Rev. Polit. Sci.* 8 (1): 297–333.

Höhne, Jan Karem, Joshua Claassen, Saijal Shahania, and David Broneske. 2025. Bots in web survey interviews: A showcase. *International Journal of Market Research* 67 (1): 3–12.

Hullman, Jessica, David Broska, Huaman Sun, and Aaron Shaw. 2026. *This human study did not involve human subjects: Validating LLM simulations as behavioral evidence.*

Kane, John V., and Jason Barabas. 2019. No Harm in Checking: Using Factual Manipulation Checks to Assess Attentiveness in Experiments. *American Journal of Political Science* 63 (1): 234–249.

Kosmyna, Nataliya, Eugene Hauptmann, Ye Tong Yuan, Jessica Situ, Xian-Hao Liao, Ashly Vivian Beresnitzky, Iris Braunstein, and Pattie Maes. 2025. *Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task.*

Lin, Gechun. 2025. Using cross-encoders to measure the similarity of short texts in political science. *American Journal of Political Science* 69 (4): 1600–1616.

Panizza, Folco, Yara Kyrychenko, and Jon Roozenbeek. 2026. Survey-taking AI tools surpass human abilities. Here's what we can do about it. *Nature* 650 (8101): 293–295.

Park, Joon Sung, Carolyn Q Zou, Aaron Shaw, Benjamin Mako Hill, Carrie J Cai, Meredith Ringel Morris, Robb Willer, Percy Liang, and Michael S Bernstein. 2025. Simulating Human Behavior with AI Agents.

Pinzón, Natalia, Vikram Koundinya, Ryan E. Galt, William O'R Dowling, Marcela Baukloh, Namah C. Taku-Forchu, Tracy Schohr, et al. 2024. AI-powered fraud and the erosion of online survey integrity: an analysis of 31 fraud detection strategies. *Frontiers in Research Metrics and Analytics* 9.

Rheault, Ludovic, and Christopher Cochrane. 2020. Word Embeddings for the Analysis of Ideological Placement in Parliamentary Corpora. *Political Analysis* 28 (1): 112–133.

Rodriguez, Pedro L., Arthur Spirling, and Brandon M. Stewart. 2023. Embedding Regression: Models for Context-Specific Description and Inference. *American Political Science Review* 117 (4): 1255–1274.

Sarstedt, Marko, Susanne J. Adler, Lea Rau, and Bernd Schmitt. 2024. Using large language models to generate silicon samples in consumer and marketing research: Challenges, opportunities, and guidelines. *Psychology & Marketing* 41 (6): 1254–1270.

Stagnaro, Michael Nicholas, James Druckman, Antonio Alonso Arechar, Robb Willer, DG Rand, et al. 2024. Representativeness versus attentiveness: a comparison across nine online survey samples.

Suguri Motoki, Fabio Yoshio, Januário Monteiro, Ricardo Malagueño, and Victor Rangel. 2024. *Large Language Models in Survey Research: Generating Synthetic Data and Unlocking New Possibilities.* SSRN Scholarly Paper. Rochester, NY.

Terry, John, Gerald Strait, Steve Alsarraf, Emily Weinmann, and Allison Waychoff. 2025. Artificial intelligence in scale development: evaluating AI-generated survey items against gold standard measures. *Current Psychology* 44 (20): 16339–16350.

Thomas, Kathrin. 2024. The Advent of Survey Experiments in Politics and International Relations. *Government and Opposition* 59 (1): 297–320.

Westwood, Sean J. 2025. The potential existential threat of large language models to online survey research. *Proceedings of the National Academy of Sciences of the United States of America* 122 (47): e2518075122.

Westwood, Sean J., and Samuel Frederick. 2026. Reply to Van der Stigchel et al.: Empirical evidence that AI survey contamination is real and substantial. *Proceedings of the National Academy of Sciences of the United States of America* 123 (8): e2537420123.

Yan, Duanli, Michael Fauss, Jiangang Hao, and Wenju Cui. n.d. Detection of AI-generated Essays in Writing Assessments.

Zhang, Simone, Janet Xu, and Aj Alvero. 2025. Generative AI Meets Open-Ended Survey Responses: Research Participant Use of AI and Homogenization. *Sociological Methods & Research* 54 (3): 1197–1242.

Ziegler, Jeffrey. 2022. A Text-As-Data Approach for Using Open-Ended Responses as Manipulation Checks. *Political Analysis* 30 (2): 289–297.

A Online Appendix

Contents

A.1 Extended Related Work 1
A.2 Experimental Design 2
A.3 An Integrated Toolkit for AI-Response Detection 6
A.3.1 Front-End Instrumentation and Transfer Logging 6
A.3.2 Behavioral Monitoring: Keystroke Dynamics Logging 7
A.3.3 Back-End Linguistic Analysis: AI Similarity and Originality Checks 9
A.4 Data Collected 15
A.5 A Synthetic AI Agent for Pre-Launch Survey Vulnerability Audits 16
A.6 Tables 19
A.7 Additional Results 19
A.8 Additional Metrics Classification 19

A.1 Extended Related Work

Recent discussions in survey methodology and political methodology converge on a common point: credible online data collection now depends less on any single screening device than on transparent workflows that document how suspicious cases are identified and handled. Reviews of survey fraud show that the usable yield of low-barrier web recruitment has deteriorated sharply and that standard defensive batteries are readily bypassed (Pinzón et al. 2024). Work in political methodology similarly shows that explicit combinations of automated and human-coded checks can recover much of the data quality achieved by intensive supervision (Cohen and Warner 2021), while studies of targeted populations document how adversarial respondents learn to defeat simple screeners (Bell and Gift 2023). The lesson is not simply that online data are noisy. It is that valid inference increasingly depends on observable, reviewable quality-control procedures.

AI assistance sharpens that requirement because it introduces a different kind of measurement failure. One useful distinction is between deception and delegation: sometimes no real respondent stands behind the record, and sometimes a real respondent is present but outsources part of the task to AI. These cases call for overlapping, but not identical, responses. Classic fraud and satisficing often degrade answers in visible ways, such as through speeding, straightlining, contradiction, or nonsense. AI-assisted delegation can do the opposite: it can improve the visible product while weakening the link between the response and the respondent's own reasoning. That matters for the open-ended tasks political scientists use to evaluate learning, comprehension, attribution, or justification. The measurement problem is therefore not only whether a respondent is human, but also whether the observed answer was produced under conditions consistent with the construct being measured (Zhang, Xu, and Alvero 2025; Westwood 2025; Westwood and Frederick 2026). Not every instance of AI use is equally damaging, but when a task is meant to capture unaided cognition, authorship becomes part of measurement.

The emerging literature has made important progress, but most contributions emphasize one part of the problem:

- **Survey fraud and data-quality audits:** This work shows that low-barrier online samples are increasingly vulnerable and that simple defensive batteries are often ineffective. Its main contribution is to document the scale of the threat and the need for layered quality control (Pinzón et al. 2024; Bell and Gift 2023).

- **Behavioral and paradata-based detection:** These studies focus on process indicators such as response speed, keystroke dynamics, paste behavior, and other paradata to identify copied or nonoriginal text. They show that writing traces can be highly informative, especially when the response was transferred or transcribed rather than composed in the survey box (Westwood and Frederick 2026; Deane et al. 2026; Diedenhofen and Musch 2017; Crossley et al. 2024).

- **Text-based or classifier-based AI detection:** This line of work tries to distinguish human from machine-generated responses using linguistic features. These tools can perform well in bounded settings, but often depend on the prompt, corpus, or model family used for training, which raises concerns about portability across survey contexts (Claassen et al. 2026; Zhang, Xu, and Alvero 2025).

- **Agent-vulnerability studies:** Recent work shows that autonomous AI respondents

can pass standard screeners, maintain coherent personas, and generate plausible answers at scale. The key contribution here is to demonstrate that the threat is no longer hypothetical and that standard attention checks are not designed for this kind of contamination (Westwood 2025; Panizza, Kyrychenko, and Roozenbeek 2026).

- **LLMs as substitutes for respondents or survey-design tools:** An adjacent literature asks when LLMs can stand in for human respondents, generate synthetic samples, or help evaluate survey items. Its focus is calibration, substitution, and design assistance rather than authentication of human-authored responses (Hullman et al. 2026; Argyle et al. 2023; Suguri Motoki et al. 2024; Fuchs, Haensch, and Weber 2026; Terry et al. 2025).

Our contribution differs from these approaches in several ways. First, we do not study only whether a response *looks* machine generated; we study whether it was produced under authorship conditions consistent with the construct the survey is meant to capture. Second, we do not rely on a single family of signals. Instead, we combine preventive design, process paradata, prompt-specific AI benchmarks, peer-distinctiveness measures, evaluator-based assessments, and, when applicable, source-text alignment in one auditable framework. Third, our approach is explicitly task-specific rather than a universal black-box detector: similarity is evaluated against benchmark AI output generated for the exact prompt, article, and instructions used in the study. Fourth, we validate the framework using experimentally assigned and directly observed AI access embedded inside the survey environment, rather than relying only on post hoc suspicion. Finally, we use a synthetic AI agent not as a substitute for human subjects, but as a benchmark and prefield auditing tool that helps diagnose survey vulnerability before launch.

For these reasons, the paper's contribution is broader than a better detector of AI text. It provides a survey-embedded authentication workflow for researchers who need to document not only the final product of responding, but also the conditions under which that product was generated.

A.2 Experimental Design

This study validates a survey-embedded approach to detecting AI-assisted responding in open-ended tasks. The design is structured to resemble a realistic paid task environment while allowing us to observe both the final responses and the process through which they are produced.

Recruitment and cover story. Participants were recruited from an online labor platform (CloudResearch Connect) and told they were completing a short paid writing task as part of a project to improve written materials. The task was framed as routine editing work rather than as a study about AI. This framing reduces demand effects and discourages respondents from strategically altering their behavior based on the study's purpose.

Task environment. All participants completed two tasks commonly found in online work: (i) summarizing a short article (approximately 150 words) and (ii) proposing a concise title. Instructions emphasized clarity, accuracy, and the use of participants' own

words. After submission, participants answered a comprehension question to ensure engagement with the source text.

To avoid confounding by content, participants were randomly assigned one of several short factual passages covering neutral topics. This ensures that differences across conditions are not driven by a specific article.

Treatment conditions and framing. The experimental manipulation varies whether participants have access to an AI writing assistant and how that access is described.

In the control condition, participants were explicitly told that external AI tools were not allowed and that submissions would be screened for AI use. This restriction was reinforced by disabling copy-paste functionality within the survey interface.

In the AI-access conditions, participants were told that a writing support tool was available to help them complete the task. The tool was presented as part of an effort to improve workflow efficiency and assist workers in producing higher-quality output. This framing mirrors how such tools are commonly introduced in real work environments.

In one condition, the tool was additionally described as part of a longer-term effort to automate similar tasks. This variation allows us to distinguish between immediate exposure to AI assistance and awareness of its broader implications.

Embedded AI tool. Rather than allowing respondents to rely on external AI platforms, we built a custom AI assistant for this study and embedded it directly into the task environment. The tool was implemented as a Streamlit application and connected to OpenAI's `gpt-4o-mini` model through the API. Embedding the assistant in this way allowed respondents to access AI support without leaving the survey workflow, while also allowing us to observe and log their interactions with the tool.

Participants in the AI conditions could paste the article into the application and receive an initial summary. They could then request up to three rounds of revisions by typing additional instructions, with the model returning an updated version after each round. This setup was designed to resemble a realistic writing-assistance tool while keeping all AI use inside a controlled interface.

The visible interface was kept constant across AI conditions. What changed across conditions was the prompt used to govern the model's behavior. In the regular-quality condition, `gpt-4o-mini` was instructed to generate a summary of about 150 words that reflected the core elements emphasized in the task instructions, including the main idea of the text, the key information, and its significance. The model was also instructed to remain faithful to the source text and to rely on its own wording rather than reproducing the article verbatim. In the low-quality condition, the same model was prompted to produce deliberately weak assistance: shorter and less complete summaries, inconsistent style, omission of central points, and occasional irrelevant or inaccurate details. Participants in both conditions could still request revisions, but in the low-quality condition the model continued to respond according to the degraded prompt. Figures A.1a and A.1b present examples of the two versions.

Embedding the tool directly in the study served an important methodological purpose. It created a treatment in which AI assistance was available as part of the task environment rather than through an external and unobserved platform, and it allowed us to collect detailed logs of tool use. At the session level, we recorded the article text pasted into

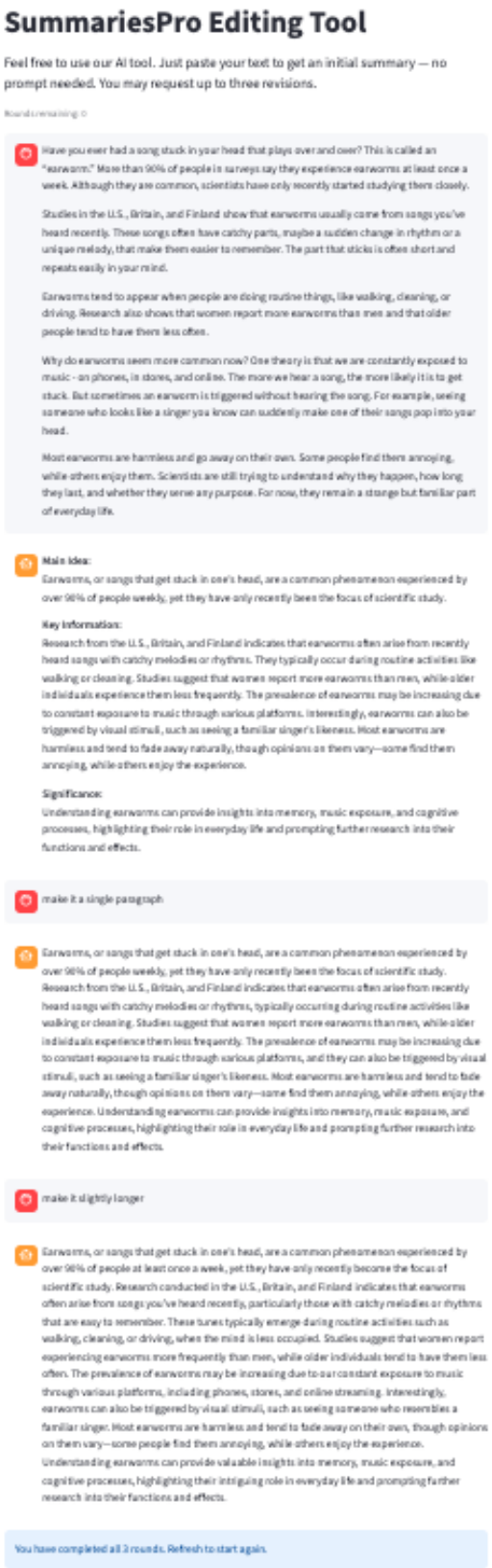
the app, the model's outputs, respondents' revision requests, and basic timing metadata. This makes it possible to distinguish assignment to AI access from actual interaction with the tool and to analyze not only the final response submitted by the participant, but also the process through which AI assistance was used.

Thus, a key feature of the design is that AI assistance is *observed* rather than inferred. Because the tool is embedded within the survey, we record all interactions, including the text entered, the model's outputs, and revision requests. This allows us to distinguish between access to AI and actual usage behavior.

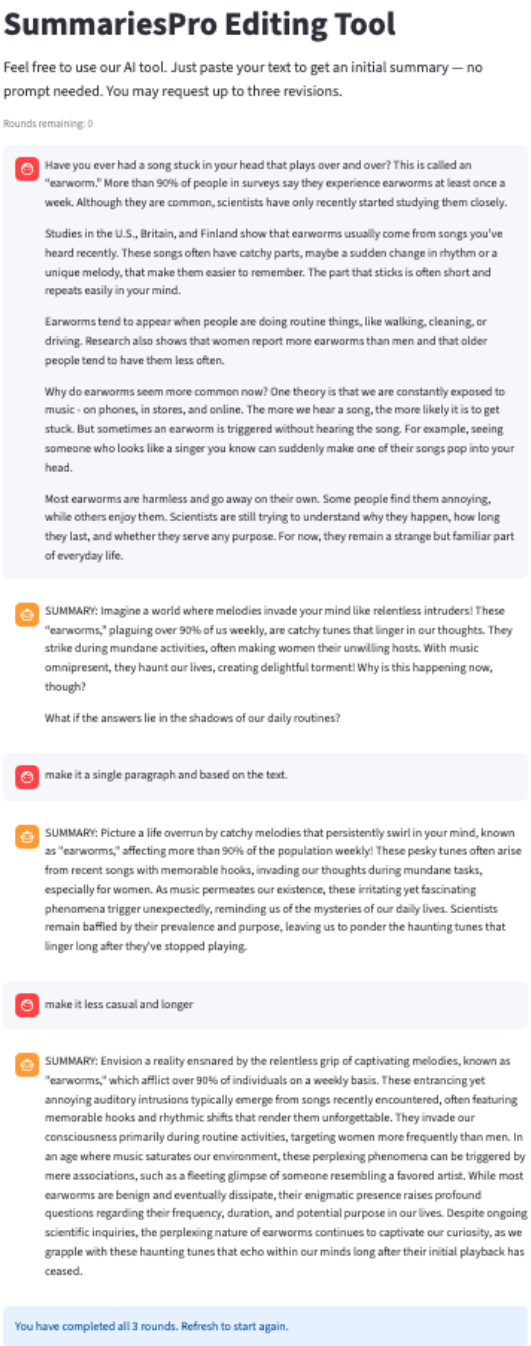
Behavioral instrumentation. The survey interface was instrumented to capture detailed process data, including keystrokes, deletions, pauses, paste attempts, and timing. These measures provide direct evidence on how responses are produced. Human-authored responses typically involve iterative drafting and revision, whereas AI-assisted responses tend to exhibit more linear production patterns or signs of transfer.

Outcome logic. The design allows us to compare both the content of responses and the process that generated them. On the textual side, we evaluate similarity to prompt-specific AI benchmarks, alignment with the source article, and distinctiveness relative to peer responses. On the behavioral side, we examine indicators such as keystroke activity, editing intensity, and production time.

Together, these measures allow us to identify whether AI assistance leaves a consistent and detectable signature, even when the final response appears fluent and relevant.



(a) Regular Quality



(b) Low Quality

Figure A.1: Examples of interaction with the Embedded AI App across conditions

A.3 An Integrated Toolkit for AI-Response Detection

This appendix documents the implementation of the three-layer toolkit summarized in Figure A.2. We report the survey-side instrumentation and the back-end Python transformations used to convert raw paradata and text into the indicators analyzed in the manuscript. The goal is to provide a transparent template that other researchers can adapt to open-ended survey tasks.

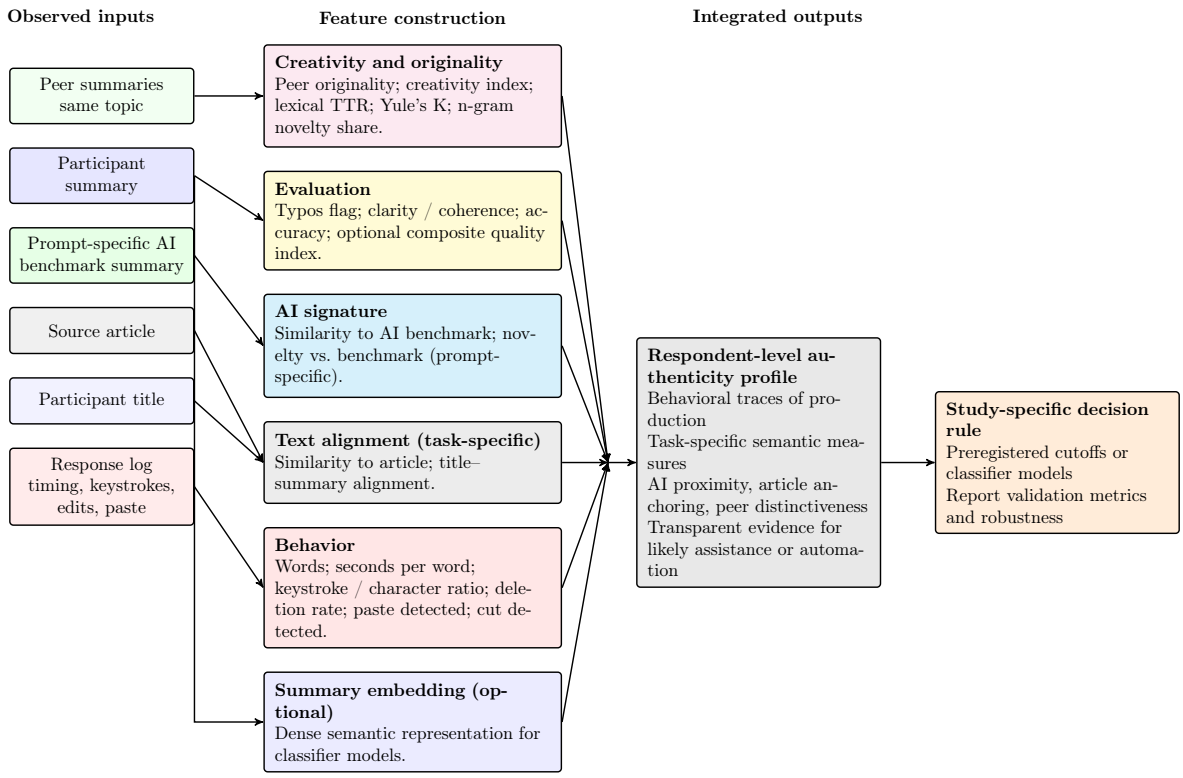


Figure A.2: Workflow for the open-ended authentication toolkit. The pipeline maps observed survey inputs into feature families—Behavior, Text alignment, AI signature, Evaluation, Creativity and originality, and an optional embedding branch—and combines them into a respondent-level authenticity profile used for study-specific classification or threshold-based flagging.

A.3.1 Front-End Instrumentation and Transfer Logging

The first layer instruments the response box at the point of entry. The client-side script initializes embedded variables for the keystroke log, character count, first-focus timestamp, total keypress count, and paste-related shortcuts. This step makes transfer behavior observable at the moment respondents write, and it creates the raw paradata used to construct the downstream behavioral indicators. The Qualtrics implementation is shown in Figure A.3.

Figure A.3: Qualtrics code for textbox-level keystroke instrumentation

```
Qualtrics.SurveyEngine.addOnload(function() {
    var qThis = this;

    Qualtrics.SurveyEngine.setEmbeddedData("keystrokeLog", "");
    Qualtrics.SurveyEngine.setEmbeddedData("charCount", "0");
    Qualtrics.SurveyEngine.setEmbeddedData("firstTextFocusTime", "");
    Qualtrics.SurveyEngine.setEmbeddedData("totalKeyPresses", "0");
    Qualtrics.SurveyEngine.setEmbeddedData("pasteKeyCount", "0");

    var inputField = qThis.getQuestionContainer().querySelector('textarea');

    if (!inputField) {
        console.error("Textarea not found!");
        return;
    }

    var keystrokeData = [];
    var totalKeyPresses = 0;
    var pasteKeyCount = 0;

    inputField.addEventListener('keydown', function(event) {
        var key = event.key;
        var timestamp = Date.now();

        totalKeyPresses += 1;
        Qualtrics.SurveyEngine.setEmbeddedData("totalKeyPresses", totalKeyPresses);

        keystrokeData.push({key: key, timestamp: timestamp});
        Qualtrics.SurveyEngine.setEmbeddedData("keystrokeLog", JSON.stringify(keystrokeData));

        var isMac = navigator.platform.toUpperCase().indexOf('MAC') >= 0;
        if (
            (isMac && event.metaKey && key.toLowerCase() === 'v') ||
            (!isMac && event.ctrlKey && key.toLowerCase() === 'v')
        ) {
            pasteKeyCount += 1;
            Qualtrics.SurveyEngine.setEmbeddedData("pasteKeyCount", pasteKeyCount);
        }
    });

    inputField.addEventListener('input', function() {
        var currentLength = inputField.value.length;
        Qualtrics.SurveyEngine.setEmbeddedData("charCount", currentLength);
    });
});

Qualtrics.SurveyEngine.addOnReady(function() {
    var qThis = this;
    jQuery("#" + qThis.questionId + " textarea").one("focus keypress", function() {
        Qualtrics.SurveyEngine.setEmbeddedData("firstTextFocusTime", Date.now());
    });
});
```

A.3.2 Behavioral Monitoring: Keystroke Dynamics Logging

The second layer converts the raw keystroke stream into response-process indicators. Because authentic writing is usually iterative, these paradata capture drafting, correction, and backtracking that are less visible when text is copied or otherwise externally generated (Crossley et al. 2024). Word count and deletion rate are constructed as shown in Figure A.5. Keystroke-to-character ratio, paste detection, cut detection, delete counts, and longest delete runs are derived from the parsed keystroke log as shown in Figure A.4. Seconds per word is then computed from task time and final word count, as shown in Figure A.6.

Figure A.4: Python code for keystroke-to-character ratio, paste detection, and cut detection

```
PASTE_MOD_KEYS = {"Control", "Meta", "Command"}
DELETE_KEYS = {"Backspace", "Delete"}
UNDO_KEYS = {"z"}
CUT_KEYS = {"x"}
COPY_KEYS = {"c"}
PASTE_KEYS = {"v"}

def parse_log(log_text: str) -> list[dict[str, Any]]:
    if not isinstance(log_text, str) or not log_text.strip():
        return []
    try:
        parsed = json.loads(log_text)
        return parsed if isinstance(parsed, list) else []
    except Exception:
        return []

def detect_combo(events: list[dict[str, Any]], mod_keys: set[str], lower_main: set[str]) -> bool:
    if len(events) < 2:
        return False
    for first, second in zip(events[:-1], events[1:]):
        first_key = str(first.get("key", ""))
        second_key = str(second.get("key", "")).lower()
        if first_key in mod_keys and second_key in lower_main:
            return True
    return False

def count_deletions(events: list[dict[str, Any]]) -> tuple[int, int]:
    count = 0
    longest_run = 0
    run = 0
    for event in events:
        key = str(event.get("key", ""))
        if key in DELETE_KEYS:
            count += 1
            run += 1
            longest_run = max(longest_run, run)
        else:
            run = 0
    return count, longest_run

def add_keystroke_ratios(df: pd.DataFrame, prefix: str, key_col: str, char_col: str) -> pd.DataFrame:
    ratio_col = f"keystroke_char_ratio_{prefix}"
    extra_col = f"extra_keys_{prefix}"
    key_values = pd.to_numeric(df[key_col], errors="coerce")
    char_values = pd.to_numeric(df[char_col], errors="coerce")
    df[ratio_col] = key_values / (char_values + 1e-6)
    df[extra_col] = key_values - char_values
    return df

def enrich_from_log(df: pd.DataFrame, prefix: str, log_col: str, key_col: str, char_col: str) -> pd.DataFrame:
    paste_flag_col = f"paste_detected_{prefix}"
    undo_flag_col = f"undo_detected_{prefix}"
    cut_flag_col = f"cut_detected_{prefix}"
    copy_flag_col = f"copy_detected_{prefix}"
    delete_count_col = f"delete_count_{prefix}"
    delete_run_col = f"delete_longest_run_{prefix}"

    paste_flags = []
    undo_flags = []
    cut_flags = []
    copy_flags = []
    delete_counts = []
    delete_runs = []

    for log_text in df[log_col].fillna(""):
        events = parse_log(log_text)
        paste_flags.append(detect_combo(events, PASTE_MOD_KEYS, PASTE_KEYS))
        undo_flags.append(detect_combo(events, PASTE_MOD_KEYS, UNDO_KEYS))
        cut_flags.append(detect_combo(events, PASTE_MOD_KEYS, CUT_KEYS))
        copy_flags.append(detect_combo(events, PASTE_MOD_KEYS, COPY_KEYS))
        delete_count, delete_run = count_deletions(events)
        delete_counts.append(delete_count)
        delete_runs.append(delete_run)

    df[paste_flag_col] = paste_flags
    df[undo_flag_col] = undo_flags
    df[cut_flag_col] = cut_flags
    df[copy_flag_col] = copy_flags
    df[delete_count_col] = delete_counts
    df[delete_run_col] = delete_runs
    return add_keystroke_ratios(df, prefix, key_col, char_col)

df = enrich_from_log(
    df,
    prefix="summary",
    log_col="keystrokeLog",
    key_col="totalKeyPresses",
    char_col="charCount",
)

if {"delete_count_summary", "summary_char_count_tracked"}.issubset(df.columns):
    derived_summary_rate = df["delete_count_summary"] / df["summary_char_count_tracked"].replace(0, np.nan)
    df["summary_deletion_rate"] = (
        df.get("summary_deletion_rate", pd.Series(index=df.index, dtype=float))
        .fillna(derived_summary_rate)
        .fillna(0)
    )
```

Figure A.5: Python code for word count and deletion rate

```
def _count_words(series: pd.Series) -> pd.Series:
    text = series.fillna("").astype(str).str.strip()
    counts = text.str.split().str.len()
    return counts.where(text.ne(""), np.nan)

survey["summary_words"] = _count_words(survey["summary_text"])
survey["summary_char_count_tracked"] = pd.to_numeric(survey["charcount"], errors="coerce")
survey["summary_keypresses"] = pd.to_numeric(survey["totalkeypresses"], errors="coerce")

survey["summary_deletion_rate"] = np.where(
    survey["summary_keypresses"] > 0,
    np.maximum(survey["summary_keypresses"] - survey["summary_char_count_tracked"], 0)
    / survey["summary_keypresses"]
    * 100,
    np.nan,
)
```

Figure A.6: Python code for seconds per word

```
SUMMARY_WORD_TIME_FEATURE = "task_page_submit_seconds_per_summary_word"

valid_summary_words = pd.to_numeric(full_df["summary_words"], errors="coerce").replace(0, np.nan)
full_df[SUMMARY_WORD_TIME_FEATURE] = pd.to_numeric(
    full_df["task_page_submit_seconds"],
    errors="coerce",
) / valid_summary_words
```

A.3.3 Back-End Linguistic Analysis: AI Similarity and Originality Checks

The third layer analyzes the submitted text itself. After collection, each response is compared to a prompt-specific AI benchmark, the source article, and the pool of peer responses written about the same article. This step creates semantic and lexical indicators that complement the behavioral trace.

Similarity to AI baseline For each task, we generate a benchmark answer from a large language model using the same prompt shown to respondents. We then embed the participant summary and the benchmark in a common vector space and compute cosine similarity between them. The implementation is shown in Figure A.7. Higher values indicate that the response is closer to the benchmark output the model would produce for that prompt.

Figure A.7: Python code for similarity to AI baseline

```
def cosine(a: np.ndarray, b: np.ndarray) -> float:
    if a is None or b is None:
        return np.nan
    na = np.linalg.norm(a)
    nb = np.linalg.norm(b)
    if na == 0 or nb == 0:
        return np.nan
    return float(np.dot(a, b) / (na * nb))

def build_embedding_matrices(
    df: pd.DataFrame,
    baselines: dict[str, dict[str, str]],
    embedding_provider: str,
    embedding_model: str,
    api_key: str | None = None,
    allow_local_embedding_fallback: bool = False,
) -> tuple[np.ndarray, np.ndarray, dict[str, np.ndarray], dict[str, np.ndarray], dict[str, np.ndarray], str]:
    summary_texts = sanitize_series(df["summary_text"]).tolist()
    title_texts = sanitize_series(df["title_text"]).tolist()
    reference_topic_order = list(TOPIC_TEXTS.keys())
    reference_texts = [TOPIC_TEXTS[key] for key in reference_topic_order]
    ai_title_texts = [baselines[key]["title"] for key in reference_topic_order]
    ai_summary_texts = [baselines[key]["summary"] for key in reference_topic_order]

    backend_label = embedding_provider
    if embedding_provider == "openai":
        try:
            client = build_openai_client(api_key=api_key)
            summary_embeddings = embed_texts_openai(summary_texts, embedding_model, client)
            title_embeddings = embed_texts_openai(title_texts, embedding_model, client)
            ref_matrix = embed_texts_openai(reference_texts, embedding_model, client)
            ai_title_matrix = embed_texts_openai(ai_title_texts, embedding_model, client)
            ai_summary_matrix = embed_texts_openai(ai_summary_texts, embedding_model, client)
            backend_label = f"openai:{embedding_model}"
        except Exception:
            if not allow_local_embedding_fallback:
                raise
            embedding_provider = "local_tfidf"

    if embedding_provider == "local_tfidf":
        corpus = [
            *summary_texts,
            *title_texts,
            *reference_texts,
            *ai_title_texts,
            *ai_summary_texts,
        ]
        embedder = LocalTfidfEmbedder(corpus_texts=corpus)
        summary_embeddings = embedder.embed(summary_texts)
        title_embeddings = embedder.embed(title_texts)
        ref_matrix = embedder.embed(reference_texts)
        ai_title_matrix = embedder.embed(ai_title_texts)
        ai_summary_matrix = embedder.embed(ai_summary_texts)
        backend_label = "local_tfidf"

    ref_embeds = {reference_topic_order[i]: ref_matrix[i] for i in range(len(reference_topic_order))}
    ai_title_emb_by_topic = {reference_topic_order[i]: ai_title_matrix[i] for i in range(len(reference_topic_order))}
    ai_summary_emb_by_topic = {reference_topic_order[i]: ai_summary_matrix[i] for i in range(len(reference_topic_order))}
    return summary_embeddings, title_embeddings, ai_title_emb_by_topic, ai_summary_emb_by_topic, ref_embeds, backend_label

df["cosine_summary_to_AI"] = [
    cosine(summary_embeddings[i], ai_summary_emb_by_topic.get(df.iloc[i]["article_topic_key"]))
    for i in range(len(df))
]
df["AI_summary"] = df["article_topic_key"].map(lambda key: baselines.get(key, {}).get("summary", "")).fillna("")
```

Originality and Faithfulness We supplement benchmark similarity with measures of originality and topical adherence. Peer originality is computed as the topic-standardized distance between a respondent's summary and the centroid of all summaries on the same topic; the implementation is shown in Figure A.8. We also compute lexical TTR (Figure A.10), Yule's K (Figure A.9), and the share of bigrams and trigrams not found in the article text (Figure A.11). These features help distinguish on-topic paraphrasing from responses that reproduce benchmark or source language too closely (Kosmyna et al. 2025).

Figure A.8: Python code for peer originality

```
def cosine_dist(a: np.ndarray, b: np.ndarray) -> float:
    value = cosine(a, b)
    return np.nan if np.isnan(value) else 1.0 - value

def zscore_by_group(values: np.ndarray, groups: List[str]) -> np.ndarray:
    values = np.array(values, dtype=float)
    groups = np.array(groups)
    out = np.full_like(values, np.nan)
    for group in np.unique(groups):
        idx = np.where(groups == group)[0]
        sub = values[idx]
        mu = np.nanmean(sub)
        sd = np.nanstd(sub)
        out[idx] = 0.0 if sd == 0 or np.isnan(sd) else (sub - mu) / sd
    return out

topic_to_rows: dict[str, list[int]] = {}
for i, topic in enumerate(topic_keys):
    topic_to_rows.setdefault(topic, []).append(i)

topic_centroid: dict[str, np.ndarray | None] = {}
for topic, idxs in topic_to_rows.items():
    vecs = summary_embeddings[idxs]
    topic_centroid[topic] = None if len(vecs) == 0 else np.nanmean(vecs, axis=0)

peer_dist = []
for i, topic in enumerate(topic_keys):
    centroid = topic_centroid.get(topic)
    summary_vec = summary_embeddings[i]
    peer_dist.append(np.nan if centroid is None else cosine_dist(summary_vec, centroid))
combined["peer_dist_summary_fullsample"] = peer_dist

combined["S_peer_orig_fullsample"] = zscore_by_group(
    combined["peer_dist_summary_fullsample"].values,
    topic_keys,
)
```

Figure A.9: Python code for Yule's K

```
def simple_tokens(text: str) -> List[str]:
    if not isinstance(text, str):
        return []
    return re.findall(r"[a-zA-Z0-9']+", text.lower())

def yules_k(tokens: List[str]) -> float:
    if not tokens:
        return np.nan
    freqs = Counter(tokens)
    N = sum(freqs.values())
    M2 = sum(value * value for value in freqs.values())
    if N <= 1:
        return np.nan
    return 1e4 * (M2 - N) / (N * N)

tokens = simple_tokens(df.iloc[i][sum_col])
lex_yule.append(yules_k(tokens))

df["lex_yule"] = lex_yule
```

Figure A.10: Python code for TTR

```
def simple_tokens(text: str) -> List[str]:
    if not isinstance(text, str):
        return []
    return re.findall(r"[a-zA-Z0-9']+", text.lower())

def ttr(tokens: List[str]) -> float:
    if not tokens:
        return np.nan
    return len(set(tokens)) / len(tokens)

tokens = simple_tokens(df.iloc[i][sum_col])
lex_ttr.append(ttr(tokens))

df["lex_ttr"] = lex_ttr
```

Figure A.11: Python code for N-gram novelty

```
def simple_tokens(text: str) -> List[str]:
    if not isinstance(text, str):
        return []
    return re.findall(r"[a-zA-Z0-9']+", text.lower())

def ngrams(seq: List[str], n: int) -> List[Tuple[str, ...]]:
    return [tuple(seq[i : i + n]) for i in range(len(seq) - n + 1)] if n > 0 else []

def build_article_ngram_sets(topic_texts: Dict[str, str]) -> Dict[str, Dict[int, set]]:
    store = {}
    for topic, text in topic_texts.items():
        tokens = simple_tokens(text)
        store[topic] = {
            2: set(ngrams(tokens, 2)),
            3: set(ngrams(tokens, 3)),
        }
    return store

art_ngs = build_article_ngram_sets(topic_texts)

tokens = simple_tokens(df.iloc[i][sum_col])
bigs = set(ngrams(tokens, 2))
tris = set(ngrams(tokens, 3))
ref_bigs = art_ngs.get(topic, {}).get(2, set())
ref_tris = art_ngs.get(topic, {}).get(3, set())

def share_novel(myset: set, refset: set) -> float:
    return np.nan if len(myset) == 0 else len(myset - refset) / len(myset)

ngram_novel_share.append(np.nanmean([share_novel(bigs, ref_bigs), share_novel(tris, ref_tris)]))

df["ngram_novel_share"] = ngram_novel_share
```

These originality components are aggregated into a composite Creativity Index after topic-specific standardization. The index combines novelty relative to the AI benchmark, article faithfulness, title-summary alignment, lexical diversity, n-gram novelty, and peer originality, with the weighting rule shown in Figure A.12. Higher values indicate responses that remain on topic while using more distinctive language and structure.

Figure A.12: Python code for Creativity index

```
weights = {
    "S_novel_summary_mm": 0.30,
    "S_novel_title_mm": 0.10,
    "S_faith_summary": 0.20,
    "S_align": 0.10,
    "S_lexdiv": 0.10,
    "S_ngram_novel": 0.10,
    "S_peer_orig": 0.10,
}

combined["Creativity_Index_fullsample"] = (
    weights["S_novel_summary_mm"] * combined["S_novel_summary_mm_fullsample"]
    + weights["S_novel_title_mm"] * combined["S_novel_title_mm_fullsample"]
    + weights["S_faith_summary"] * combined["S_faith_summary_fullsample"]
    + weights["S_align"] * combined["S_align_fullsample"]
    + weights["S_lexdiv"] * combined["S_lexdiv_fullsample"]
    + weights["S_ngram_novel"] * combined["S_ngram_novel_fullsample"]
    + weights["S_peer_orig"] * combined["S_peer_orig_fullsample"]
)
```

AI-as-Judge Evaluation Finally, we use an LLM as evaluator rather than generator. Each participant summary is scored against the reference article on clarity and coherence, factual accuracy, use of own words, and typographical quality. This evaluator step provides a structured assessment that is distinct from the benchmark-similarity measures and is useful when researchers want interpretable ratings alongside automated text features (Kosmyna et al. 2025). The evaluation prompt and API specification are reported in Figures A.13 and A.14.

Figure A.13: AI-as-judge evaluation prompt

```
SYSTEM_PROMPT = (
    'You are a careful evaluator of respondent-written article summaries. '
    'Return valid JSON only.'
)

def build_evaluation_prompt(reference_text: str, respondent_summary: str) -> str:
    return f"""
You are an evaluator. Assess a summary written by a survey respondent.

They were instructed to include:
- Main Idea: central topic or purpose of the article.
- Key Information: most important points or findings.
- Significance: why the information is important or useful.

Evaluation criteria (score each 1-5):
1) Clarity & Coherence
2) Accuracy (faithfulness to the article's main idea, key points, and significance)
3) Use of Own Words (avoid copying; paraphrase appropriately)

Also run a quick spelling and grammar pass and report whether there are typos.

Output JSON ONLY, matching this exact schema and key order:
{
  "Clarity & Coherence": <1-5 integer>,
  "Accuracy": <1-5 integer>,
  "Use of Own Words": <1-5 integer>,
  "Typos_Flag": <true|false>,
  "Typos_Notes": "<one short sentence on any errors>",
  "Notes": "<2-4 concise sentences: what worked, what to improve>"
}

Reference article:
[[REFERENCE]]
{reference_text}
[[/REFERENCE]]

Respondent summary:
[[SUMMARY]]
{respondent_summary}
[[/SUMMARY]]

Use only the reference article above to judge accuracy.
If the summary is blank or off-topic, assign 1s across criteria and explain briefly in Notes.
Return valid JSON and nothing else.
""".strip()
```

Figure A.14: OpenAI API specification for AI-as-judge evaluation

```
MODEL_NAME = 'gpt-4o-mini'

def call_openai(client, prompt: str, model: str, max_retries: int = 5, base_sleep: float = 2.0) -> str:
    last_err = None
    for attempt in range(max_retries):
        try:
            response = client.chat.completions.create(
                model=model,
                messages=[
                    {'role': 'system', 'content': SYSTEM_PROMPT},
                    {'role': 'user', 'content': prompt},
                ],
                temperature=0.0,
            )
            text = response.choices[0].message.content or ''
            return text.strip()
        except Exception as err:
            last_err = err
            sleep_for = base_sleep * (2 ** attempt) + 0.25 * attempt
            time.sleep(min(30.0, sleep_for))

    raise RuntimeError(f'OpenAI API failed after retries: {last_err}')
```

Taken together, the three layers produce a respondent-level authenticity profile rather than a single binary label. This is the format most useful for replication and applied survey work: researchers can inspect individual indicators, preregister thresholds, or train classifiers on the full feature set depending on the needs of the study.

A.4 Data Collected

For each participant's summary response, we recorded the full text of the answer and a detailed keystroke log (including timestamps for each key event). Using the toolkit's back-end analysis, we generated a set of behavioral and linguistic indicators for every response. These measures jointly capture the process of composition (through keystroke dynamics) and the content of the written text (through embedding-based and lexical analysis). Specifically, we computed:

- **Character Count:** Total number of characters in the final submitted response.
- **Keystroke Count:** Total number of keys pressed, including letters, numbers, spaces, deletions, and other editing keys.
- **Extra Keystrokes:** Difference between keystroke count and character count, indicating the amount of revision and editing (extra keystrokes = keystrokes − characters).
- **Keystroke–Character Ratio:** Ratio of total key presses to final characters (keystrokes/characters), representing the extent of typing activity per retained character.
- **Paste Event:** Binary indicator denoting whether a paste action (`Ctrl/Command+V`) occurred during text entry.

- **Cosine Similarity to AI Answer:** Semantic similarity between the participant's response and a GPT-3.5–generated reference summary, computed using embeddings from the `text-embedding-3-small` model.
- **Cosine Similarity to Article:** Similarity between the participant's response and the original article text, reflecting topical faithfulness.
- **Novelty vs. AI:** Cosine distance ($1 -$ cosine similarity) from the AI baseline, measuring how distinct the participant's phrasing and ideas are from model-generated language.
- **Summary–Title Alignment:** Cosine similarity between a participant's summary and title embeddings, used as a measure of internal coherence.
- **Lexical Diversity:** Vocabulary richness estimated via the type–token ratio (TTR) and Yule's K statistic.
- **N-gram Novelty:** Proportion of bigrams and trigrams in the participant's response not present in the original article, capturing unique phrasing.
- **Peer Originality:** Embedding distance between a participant's summary and the centroid of all summaries written about the same article, indicating distinctiveness among peers.
- **AI-as-Judge Evaluation:** Structured ratings provided by a large language model (GPT-4) assessing clarity, coherence, accuracy, and typos.

All variables are standardized within topic to account for differences in article complexity, and they are combined in subsequent analyses to construct composite indicators such as the *Creativity Index* and overall measures of AI-likeness and response authenticity.

These measures were collected uniformly for all responses. The keystroke features and content features were then analyzed to see which best separate the two experimental groups (which serve as a proxy for likely AI use, since presumably AI-Allowed should have a higher incidence of AI-written answers). Because we experimentally manipulated one factor, we can interpret differences more cleanly than in an observational setting. In practice, a researcher using this toolkit in a real-world survey (without experimental conditions) would rely on a combination of metrics and perhaps a trained model to flag cases of AI-generated content.

A.5 A Synthetic AI Agent for Pre-Launch Survey Vulnerability Audits

An AI agent, in this setting, is a bounded software system that can perceive a survey page, represent the current task, generate answers conditional on a stable persona, and execute those answers in a browser without manual intervention. Rather than treating AI exposure as an abstract risk, researchers can use such an agent as a pre-launch stress test on their own instrument. If a controlled synthetic respondent can complete the survey reliably, answer comprehension items from on-screen context, and produce plausible open-ended text at low cost, then the instrument is vulnerable to AI-mediated responding. The

implementation described here was built for controlled benchmarking on an owned survey. Although the prototype targeted standard Qualtrics page structures, the logic is portable to other web surveys that expose ordinary HTML form elements.

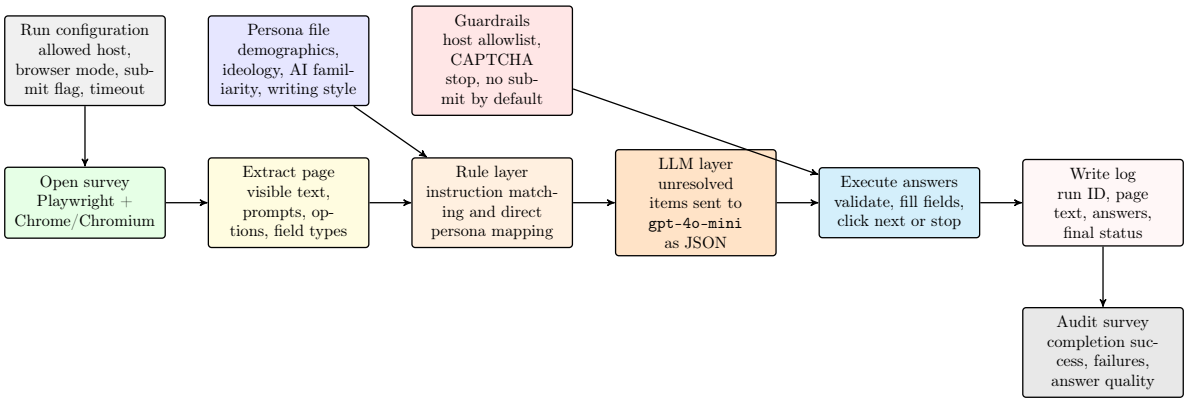


Figure A.15: Synthetic-agent audit workflow. The agent opens the survey in a controlled browser, extracts visible questions into structured fields, answers simple items with rules, sends unresolved items to `gpt-4o-mini`, executes the validated answers in the browser, and stores a full run log for pre-launch auditing.

Using the agent as a template for vulnerability detection. Researchers can use this design as a pre-launch audit protocol. First, create a small pool of realistic personas rather than a single synthetic respondent. Second, run the instrument in dry mode and inspect the logs for unsupported widgets, broken parsers, or pages that the agent cannot interpret. Third, run a small submitted batch on the pilot version of the owned survey. Fourth, inspect whether the agent can pass attention checks, answer comprehension items from on-screen text, and generate plausible open-ended responses. Finally, compare completion success, answer diversity across personas, and the substantive quality of the generated text. A survey that a low-cost agent can complete reliably under these conditions should be treated as vulnerable to AI-mediated responding before field launch, even if ordinary fraud checks would not flag those cases.

Model and runtime. The benchmark agent was implemented in Python and used the OpenAI API together with Playwright browser automation. In the configuration used for the audit, the answer model was `gpt-4o-mini`, temperature was set to 0.5, the browser ran headless by default, the timeout was 15,000 milliseconds, and the agent retained up to four prior pages of textual context when answering later items. The core software stack was: `openai>=1.0.0` for model calls, `playwright==1.58.2` for browser control, Chrome/Chromium for page execution, JSON persona and configuration files for controlled variation, and JSON log files for post-run auditing. For larger batches, the same runner could be containerized and sharded across cloud jobs, with optional artifact upload through Google Cloud Storage.

What the agent actually received. The model did not receive unconstrained browsing instructions. Instead, it received a structured payload containing: (i) persona fields

such as age group, gender, education, ethnicity, work status, occupation, ideology, AI familiarity, AI usage, description, and writing style; (ii) recent page text from earlier screens; (iii) current page text; (iv) prior answers; and (v) a list of unresolved fields with the prompt, field type, available options, and local question context. This design matters because it makes the agent auditable: researchers can see exactly what information the model had at each step.

Core instruction prompt. The system prompt supplied to the model was intentionally narrow and operational rather than conversational:

> "You are generating synthetic survey responses. Answer consistently as the provided persona. Follow the question instructions exactly. Return only valid JSON."

The response schema was similarly constrained:

```
{"answers": {"FIELD_ID": "string or list of strings"}}
```

Hybrid answer strategy. Not every answer was generated freely by the language model. Before calling the model, the agent first applied deterministic rules. If a question explicitly instructed the respondent to select a quoted option, the agent matched that option directly. If a field asked for demographics or background items represented in the persona file, the agent mapped the stored persona values onto the available response options. Only unresolved items were passed to the model. After the model responded, every returned answer was validated against the extracted response options before being entered into the survey. This hybrid design reduced unnecessary model calls and made the agent more consistent across repeated runs.

Survey parsing and supported question types. The page parser inspected visible question blocks and transformed them into a structured representation containing a field identifier, question identifier, prompt text, response type, required status, selector, response options, and local context. In the implementation used here, the agent supported five common classes of items: single-choice questions, multi-select questions, dropdown menus, text-entry boxes, and radio-matrix rows. Unsupported widgets were not silently guessed; instead, they appeared as failures or incomplete pages in the run log, which is itself useful information for a pre-launch vulnerability audit.

How the agent was run. A dry run opened the survey, answered pages, and stopped before the final submission button:

```
python -m survey_agent \
  --survey-url "YOUR_SURVEY_URL" \
  --persona-file configs/persona.json \
  --config-file configs/agent_config.json
```

Adding `-submit` allowed final submission on an owned survey. Adding `-count`, `-persona-pool-file`, and `-sleep-seconds` enabled small benchmark batches across multiple synthetic personas. A visible browser mode was also available through `-headful`, and a manual pause could be triggered at a CAPTCHA gate if the researcher wanted to inspect that failure mode.

Guardrails and audit value. Each run wrote a JSON log containing the final status, extracted page text, parsed fields, chosen answers, and error information when relevant. These safeguards are methodologically useful because the goal is not covert completion of arbitrary surveys, but controlled benchmarking of one's own instrument before deployment.

A.6 Tables

Table A.1: Summary-only coefficient regressions: AI assistance (AI_only) vs Control (refers to Figure 2)

	Dependent variable												
	Key/char	Delete rate	Paste	Words	Sec./word	Cut	AI sim.	Typos	Peer orig.	Creativ. index	Yule's K	TTR	N-gram nov.
AI assistance	-1.040***	-0.790***	0.813***	0.396***	-0.186***	-0.056	0.887***	-0.533***	-0.706***	-0.827***	-0.277***	0.148**	-0.331***
	(0.060)	(0.064)	(0.064)	(0.068)	(0.071)	(0.071)	(0.059)	(0.067)	(0.065)	(0.063)	(0.067)	(0.067)	(0.067)
Topic: Dreams	-0.055	-0.093	0.072	0.234*	-0.116	0.114	0.526***	-0.023	0.093	-0.011	0.016	-0.091	-0.316**
	(0.107)	(0.112)	(0.122)	(0.125)	(0.169)	(0.112)	(0.110)	(0.128)	(0.120)	(0.111)	(0.112)	(0.118)	(0.153)
Topic: Kindness	-0.000	-0.020	0.006	0.147	-0.082	-0.000	0.272**	0.068	0.110	0.012	0.472***	-0.468***	0.291**
	(0.106)	(0.117)	(0.115)	(0.139)	(0.164)	(0.004)	(0.107)	(0.125)	(0.123)	(0.115)	(0.144)	(0.134)	(0.136)
Topic: Music	0.142	0.153	-0.049	-0.108	-0.039	0.007	-0.157	0.248*	0.135	0.192	-0.308**	0.338**	0.108
	(0.113)	(0.125)	(0.118)	(0.125)	(0.189)	(0.009)	(0.117)	(0.131)	(0.130)	(0.122)	(0.121)	(0.131)	(0.132)
Topic: Sleep	0.036	-0.012	0.119	0.060	0.016	0.004	0.446***	0.057	0.096	-0.062	-0.298***	0.257**	0.013
	(0.105)	(0.116)	(0.117)	(0.116)	(0.178)	(0.006)	(0.126)	(0.127)	(0.120)	(0.123)	(0.090)	(0.123)	(0.147)
Topic: Smelling	0.103	0.085	-0.024	-0.026	-0.225	0.109	-0.042	-0.096	0.192	0.131	-0.030	0.109	0.508***
	(0.107)	(0.121)	(0.113)	(0.122)	(0.164)	(0.108)	(0.090)	(0.128)	(0.118)	(0.106)	(0.105)	(0.126)	(0.123)
Topic: Sugar	0.119	0.118	0.077	0.078	0.068	0.364**	-0.080	0.059	0.138	0.143	0.305***	-0.198	0.337***
	(0.116)	(0.126)	(0.114)	(0.126)	(0.171)	(0.179)	(0.090)	(0.123)	(0.120)	(0.109)	(0.111)	(0.129)	(0.124)
Constant	0.472***	0.363***	-0.439***	-0.255***	0.143	-0.060*	-0.583***	0.221**	0.246***	0.358***	0.108	-0.059	0.027
	(0.077)	(0.093)	(0.076)	(0.089)	(0.137)	(0.032)	(0.069)	(0.097)	(0.085)	(0.077)	(0.078)	(0.096)	(0.100)
N	828	828	828	828	828	828	828	828	828	828	828	828	828
Positive class N	416	416	416	416	416	416	416	416	416	416	416	416	416
Control N	412	412	412	412	412	412	412	412	412	412	412	412	412
R^2	0.275	0.162	0.169	0.050	0.017	0.017	0.265	0.078	0.127	0.178	0.098	0.075	0.092
Adj. R^2	0.268	0.155	0.162	0.042	0.008	0.009	0.258	0.070	0.120	0.171	0.090	0.067	0.085
Topic FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Notes: Standard errors in parentheses; HC1 robust SEs. All regressions include topic fixed effects, standardized outcomes, and omit the Coloring topic. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

A.7 Additional Results

Figure A.16 reports the extended benchmark coefficient plot for the original human sample, comparing each AI treatment arm separately to the same baseline control group. Positive coefficients indicate that the corresponding outcome is higher in the given AI arm than in the control group, while negative coefficients indicate lower values relative to control.

A.8 Additional Metrics Classification

Table A.2: Summary-only coefficient regressions: AI assistance and AI agent vs Baseline Control (refers to Figure 3)

Panel A. AI assistance vs Baseline Control

	Dependent variable												
	Sec./word	Words	Delete rate	Key/char	Paste	Cut	AI sim.	Typos	TTR	Yule's K	N-gram nov.	Peer orig.	Creativ. index
AI assistance	-0.186***	0.396***	-0.790***	-1.040***	0.813***	-0.056	0.887***	-0.533***	0.148**	-0.277***	-0.331***	-0.810***	-0.892***
	(0.071)	(0.068)	(0.064)	(0.060)	(0.064)	(0.071)	(0.059)	(0.067)	(0.067)	(0.067)	(0.067)	(0.064)	(0.061)
Topic: Dreams	-0.116	0.234*	-0.093	-0.055	0.072	0.114	0.526***	-0.023	-0.091	0.016	-0.316**	0.012	-0.073
	(0.169)	(0.125)	(0.112)	(0.107)	(0.122)	(0.112)	(0.110)	(0.128)	(0.118)	(0.112)	(0.153)	(0.118)	(0.113)
Topic: Kindness	-0.082	0.147	-0.020	-0.000	0.006	-0.000	0.272**	0.068	-0.468***	0.472***	0.291**	0.046	0.027
	(0.164)	(0.139)	(0.117)	(0.106)	(0.115)	(0.004)	(0.107)	(0.125)	(0.134)	(0.144)	(0.136)	(0.120)	(0.115)
Topic: Music	-0.039	-0.108	0.153	0.142	-0.049	0.007	-0.157	0.248*	0.338**	-0.308**	0.108	0.091	0.038
	(0.189)	(0.125)	(0.125)	(0.113)	(0.118)	(0.009)	(0.117)	(0.131)	(0.131)	(0.121)	(0.132)	(0.127)	(0.129)
Topic: Sleep	0.016	0.060	-0.012	0.036	0.119	0.004	0.446***	0.057	0.257**	-0.298***	0.013	0.093	-0.118
	(0.178)	(0.116)	(0.116)	(0.105)	(0.117)	(0.006)	(0.126)	(0.127)	(0.123)	(0.090)	(0.147)	(0.116)	(0.119)
Topic: Smelling	-0.225	-0.026	0.085	0.103	-0.024	0.109	-0.042	-0.096	0.109	-0.030	0.508***	0.119	0.078
	(0.164)	(0.122)	(0.121)	(0.107)	(0.113)	(0.108)	(0.090)	(0.128)	(0.126)	(0.105)	(0.123)	(0.119)	(0.104)
Topic: Sugar	0.068	0.078	0.118	0.119	0.077	0.364**	-0.080	0.059	-0.198	0.305***	0.337***	0.111	0.169
	(0.171)	(0.126)	(0.126)	(0.116)	(0.114)	(0.179)	(0.090)	(0.123)	(0.129)	(0.111)	(0.124)	(0.116)	(0.105)
Constant	0.143	-0.255***	0.363***	0.472***	-0.439***	-0.060*	-0.583***	0.221**	-0.059	0.108	0.027	0.339***	0.429***
	(0.137)	(0.089)	(0.093)	(0.077)	(0.076)	(0.032)	(0.069)	(0.097)	(0.096)	(0.078)	(0.100)	(0.082)	(0.077)
N	828	828	828	828	828	828	828	828	828	828	828	828	828
Positive class N	416	416	416	416	416	416	416	416	416	416	416	416	416
Control N	412	412	412	412	412	412	412	412	412	412	412	412	412
R^2	0.017	0.050	0.162	0.275	0.169	0.017	0.265	0.078	0.075	0.098	0.092	0.166	0.210
Adj. R^2	0.008	0.042	0.155	0.268	0.162	0.009	0.258	0.070	0.067	0.090	0.085	0.159	0.203
Topic FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Panel B. AI agent vs Baseline Control

	Dependent variable												
	Sec./word	Words	Delete rate	Key/char	Paste	Cut	AI sim.	Typos	TTR	Yule's K	N-gram nov.	Peer orig.	Creativ. index
AI agent	-1.439***	-0.073	-1.551***	-1.890***	-0.334***	-0.151**	0.978***	-1.228***	0.828***	-1.048***	-0.392***	-1.167***	-1.130***
	(0.053)	(0.062)	(0.048)	(0.026)	(0.073)	(0.075)	(0.049)	(0.060)	(0.055)	(0.053)	(0.060)	(0.048)	(0.048)
Topic: Dreams	0.054	-0.044	-0.102	-0.061	0.037	0.133	0.201*	0.035	0.001	0.077	-0.270**	0.152	0.022
	(0.073)	(0.120)	(0.074)	(0.039)	(0.104)	(0.125)	(0.103)	(0.095)	(0.104)	(0.086)	(0.132)	(0.104)	(0.103)
Topic: Kindness	-0.012	0.150	-0.107	-0.075*	0.180	-0.004	0.230***	0.084	-0.358***	0.381***	0.242**	0.038	-0.064
	(0.068)	(0.120)	(0.079)	(0.044)	(0.133)	(0.009)	(0.086)	(0.097)	(0.101)	(0.093)	(0.105)	(0.095)	(0.090)
Topic: Music	-0.049	-0.133	0.008	-0.018	0.030	0.004	-0.103	0.116	0.215**	-0.199**	0.216*	-0.031	-0.231**
	(0.067)	(0.123)	(0.088)	(0.047)	(0.108)	(0.010)	(0.098)	(0.099)	(0.101)	(0.088)	(0.120)	(0.102)	(0.102)
Topic: Sleep	0.103	-0.272**	-0.061	-0.047	0.059	0.006	0.326***	0.035	0.180*	-0.052	0.097	0.086	-0.204*
	(0.097)	(0.112)	(0.081)	(0.044)	(0.107)	(0.009)	(0.121)	(0.094)	(0.103)	(0.087)	(0.120)	(0.095)	(0.105)
Topic: Smelling	-0.084	-0.139	-0.011	-0.004	-0.042	0.117	-0.194**	-0.028	0.023	0.091	0.515***	0.153	0.093
	(0.069)	(0.118)	(0.083)	(0.043)	(0.087)	(0.115)	(0.087)	(0.096)	(0.105)	(0.089)	(0.108)	(0.093)	(0.089)
Topic: Sugar	0.139	-0.010	-0.027	-0.030	0.088	0.211	-0.020	0.134	-0.253**	0.514***	0.379***	0.077	0.011
	(0.086)	(0.108)	(0.086)	(0.049)	(0.116)	(0.149)	(0.081)	(0.096)	(0.103)	(0.097)	(0.101)	(0.094)	(0.088)
Constant	0.794***	0.104	0.926***	1.109***	0.138	0.019	-0.624***	0.645***	-0.435***	0.469***	0.051	0.595***	0.694***
	(0.064)	(0.081)	(0.078)	(0.042)	(0.091)	(0.042)	(0.059)	(0.086)	(0.071)	(0.062)	(0.082)	(0.065)	(0.062)
N	955	955	955	955	955	955	955	955	955	955	955	955	955
Positive class N	543	543	543	543	543	543	543	543	543	543	543	543	543
Control N	412	412	412	412	412	412	412	412	412	412	412	412	412
R^2	0.514	0.018	0.593	0.878	0.033	0.012	0.269	0.375	0.215	0.333	0.097	0.336	0.327
Adj. R^2	0.510	0.010	0.590	0.877	0.025	0.005	0.263	0.370	0.209	0.328	0.090	0.331	0.322
Topic FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Notes: Standard errors in parentheses; HC1 robust SEs. All regressions include topic fixed effects, standardized outcomes, and omit the Coloring topic. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

Indicator	AI only				Any AI				AI assistance or Agent			
Indicator	ROC-AUC	Avg. prec.	Recall	F1	ROC-AUC	Avg. prec.	Recall	F1	ROC-AUC	Avg. prec.	Recall	F1
Behavior												
Seconds per word	0.601	0.642	0.668	0.591	0.586	0.831	0.669	0.712	0.836	0.939	0.779	0.816
Words	0.640	0.608	0.649	0.643	0.597	0.802	0.567	0.667	0.603	0.744	0.707	0.748
Deletion rate	0.762	0.806	0.697	0.681	0.730	0.898	0.683	0.754	0.883	0.944	0.853	0.874
Keystroke / char ratio	0.767	0.820	0.567	0.702	0.734	0.904	0.513	0.669	0.896	0.959	0.802	0.883
Paste detected	0.653	0.675	0.385	0.537	0.628	0.835	0.322	0.482	0.551	0.743	0.167	0.281
Cut detected	0.499	0.501	0.995	0.669	0.499	0.750	0.995	0.855	0.502	0.700	0.998	0.824
AI signature												
Similarity to AI baseline	0.724	0.793	0.565	0.646	0.668	0.879	0.520	0.648	0.737	0.905	0.643	0.759
Evaluation												
Typos flag	0.609	0.571	0.745	0.668	0.600	0.794	0.723	0.767	0.686	0.791	0.889	0.847
Creativity and originality												
Lexical TTR (higher = more diverse)	0.549	0.531	0.543	0.547	0.565	0.780	0.557	0.654	0.655	0.807	0.569	0.662
Lexical Yule's K (lower = more diverse)	0.637	0.628	0.668	0.633	0.610	0.821	0.640	0.714	0.740	0.867	0.712	0.768
N-gram novelty share (higher = more novel)	0.608	0.609	0.490	0.542	0.558	0.793	0.449	0.575	0.623	0.794	0.589	0.683
Peer originality (higher = more original)	0.699	0.778	0.567	0.622	0.633	0.865	0.521	0.639	0.747	0.906	0.644	0.750
Creativity index (higher = more creative)	0.723	0.795	0.575	0.642	0.656	0.875	0.517	0.643	0.748	0.907	0.642	0.758

Table A.3: Standalone predictive power of indicators across AI-use classifiers

Notes: Each cell reports performance from a standalone cross-validated logistic-regression classifier estimated with a single indicator at a time. Columns report ROC-AUC, average precision, recall, and F1. *AI only* compares human `AI_only` responses to Baseline Control; *Any AI* compares all human AI-assistance arms (`AI_only`, `AI_low`, and `AI_long_term`) to Baseline Control; *AI assistance or Agent* combines human `AI_only` responses and all AI agent responses against Baseline Control. Indicators are grouped into *Behavior*, *AI signature*, *Evaluation*, and *Creativity and originality*, matching the summary-only benchmark coefficient plot. Baseline Control has $N = 412$; positive-class sample sizes are $N = 416$ for AI only, $N = 1235$ for Any AI, and $N = 959$ for AI assistance or Agent.

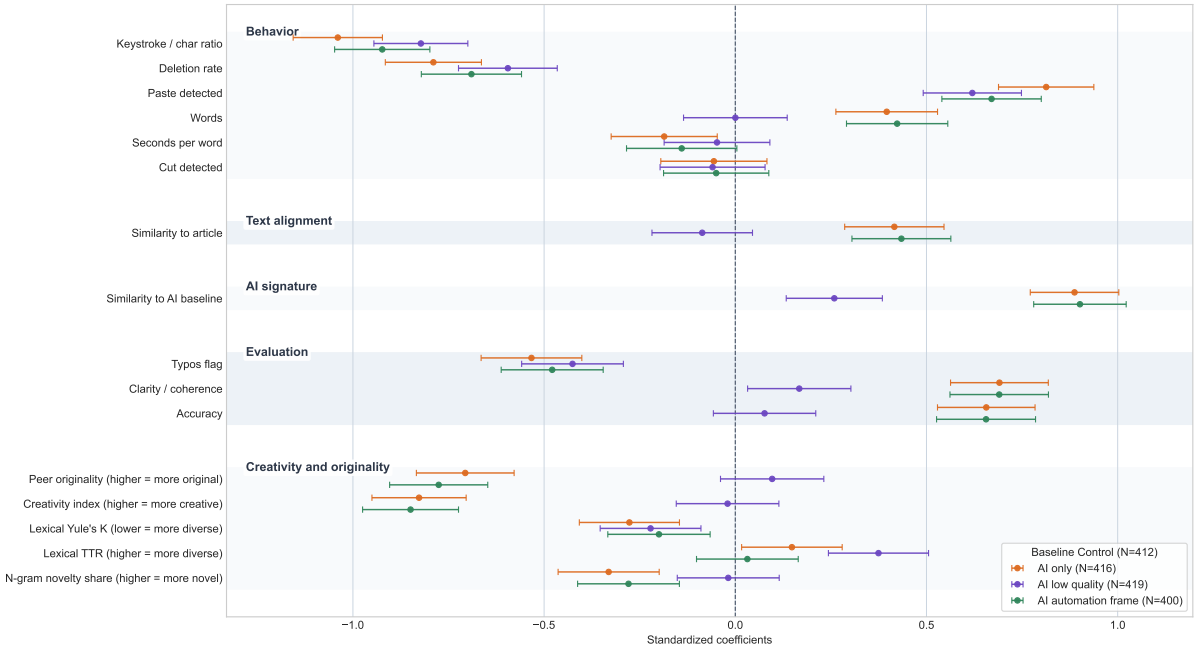


Figure A.16: Extended benchmark coefficients for human AI-treatment arms relative to baseline control.

Notes: The figure reports standardized coefficients from separate topic-adjusted OLS regressions estimated on the original human dataset, with the same baseline control group used as the reference category in each comparison. The three series correspond to `AI_only` ($n = 416$), `AI_low` ($n = 419$), and `AI_long_term` ($n = 400$), each compared against baseline control ($n = 412$). Outcomes are grouped into the same extended categories as in the main coefficient plot: *Behavior*, *Text alignment*, *AI signature*, *Evaluation*, and *Creativity and originality*. The extended specification includes *Seconds per word* in Behavior, *Similarity to article* in Text alignment, and *Clarity / coherence* and *Accuracy* in Evaluation. Whiskers denote 95% confidence intervals. Positive values indicate higher levels of the measure relative to baseline control.

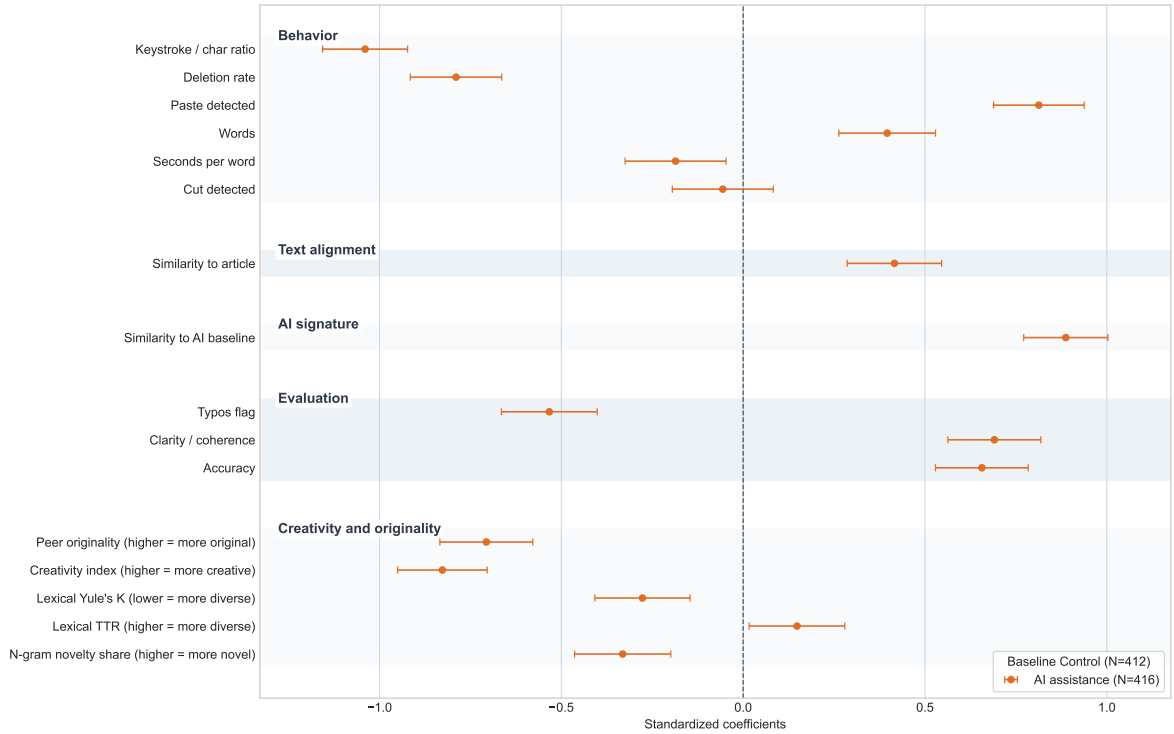


Figure A.17: Extended standardized differences in summary-task outcomes between the human AI-only treatment and the human control group.

Notes: This figure uses the same sample and specification as Figure 2: each coefficient comes from a separate topic-adjusted OLS regression with article-topic fixed effects, estimated on human `AI assisted` observations and control observations. Dependent variables are standardized, so coefficients are directly comparable across rows. Positive coefficients indicate that the AI-only group scores higher than the control group on the relevant outcome; negative coefficients indicate lower values. Relative to Figure 2, this extended specification adds a *Text alignment* category, where *Similarity to article* measures how closely the submitted summary tracks the source article, and expands the *Evaluation* category to include *Clarity / coherence* and *Accuracy*, where higher values indicate better evaluator assessments. As in Figure 2, the *Behavior* outcomes capture effort, speed, and copy/paste activity; the *AI signature* outcome captures similarity to an AI baseline; *Typos flag* is a binary indicator for whether typos were flagged; and the *Creativity and originality* outcomes capture originality, creativity, lexical diversity, and novelty.

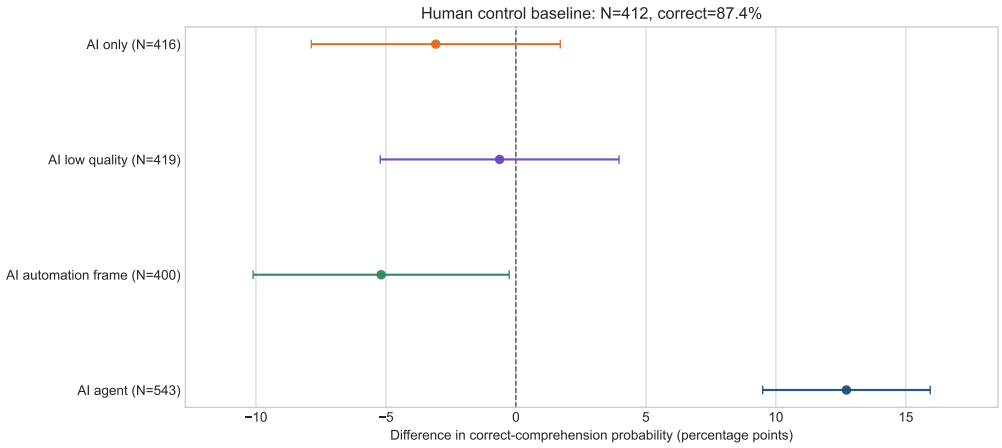


Figure A.18: Comprehension accuracy across AI conditions relative to human control.
Notes: Each point compares one AI condition with the shared human control baseline in a separate linear probability model with article-topic fixed effects. Horizontal bars show 95% HC1 confidence intervals. Coefficients are reported in percentage points, so positive values indicate higher probability of answering the comprehension question correctly than in the human control group and negative values indicate lower probability. The four contrasts are `AI_only`, `AI_low`, `AI_long_term`, and the AI-agent benchmark. The human control baseline contains $N = 412$ observations and an 87.4% correct-comprehension rate.

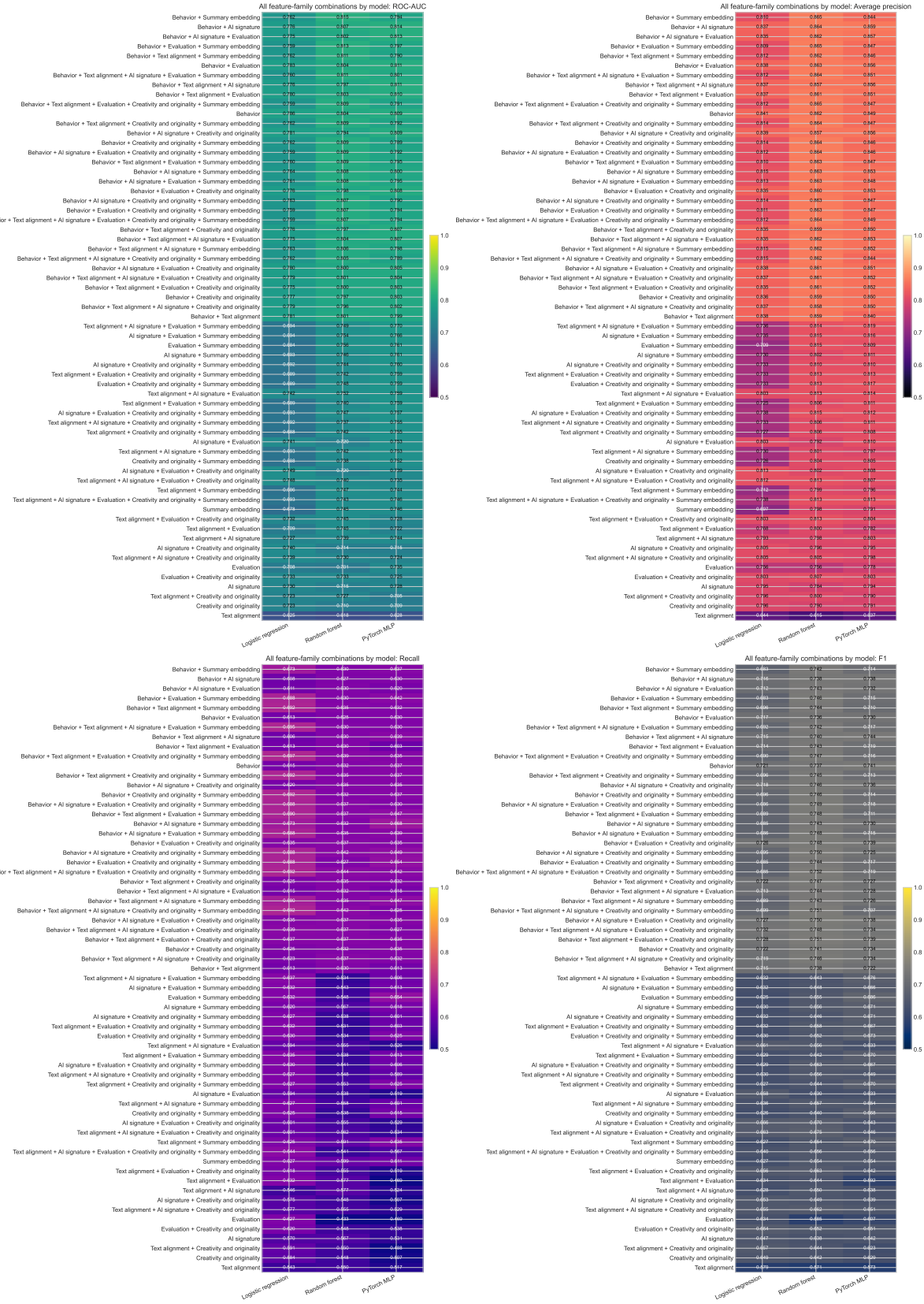


Figure A.19: AI-only assistance versus baseline control: classifier performance across feature-family combinations.

Notes: The figure reports out-of-sample classifier performance for distinguishing human responses written with AI assistance (`AI_only`) from baseline human control responses. The analysis sample contains $N = 828$ observations ($n = 416$ AI-only, $n = 412$ control). Columns correspond to logistic regression, random forest, and PyTorch MLP models. Rows correspond to all possible combinations of the feature families *Behavior*, *Text alignment*, *AI signature*, *Evaluation*, *Creativity and originality*, and *Summary embedding*. The four panels report ROC-AUC, average precision, recall, and F1. Rows are ordered by the highest ROC-AUC achieved by any model for that feature-family combination. The *Summary embedding* family is based on a 384-dimensional sentence-transformer representation of the written summary (`all-MiniLM-L6-v2`). In this comparison, the strongest-performing bundle combines *Behavior* and *Summary embedding*.

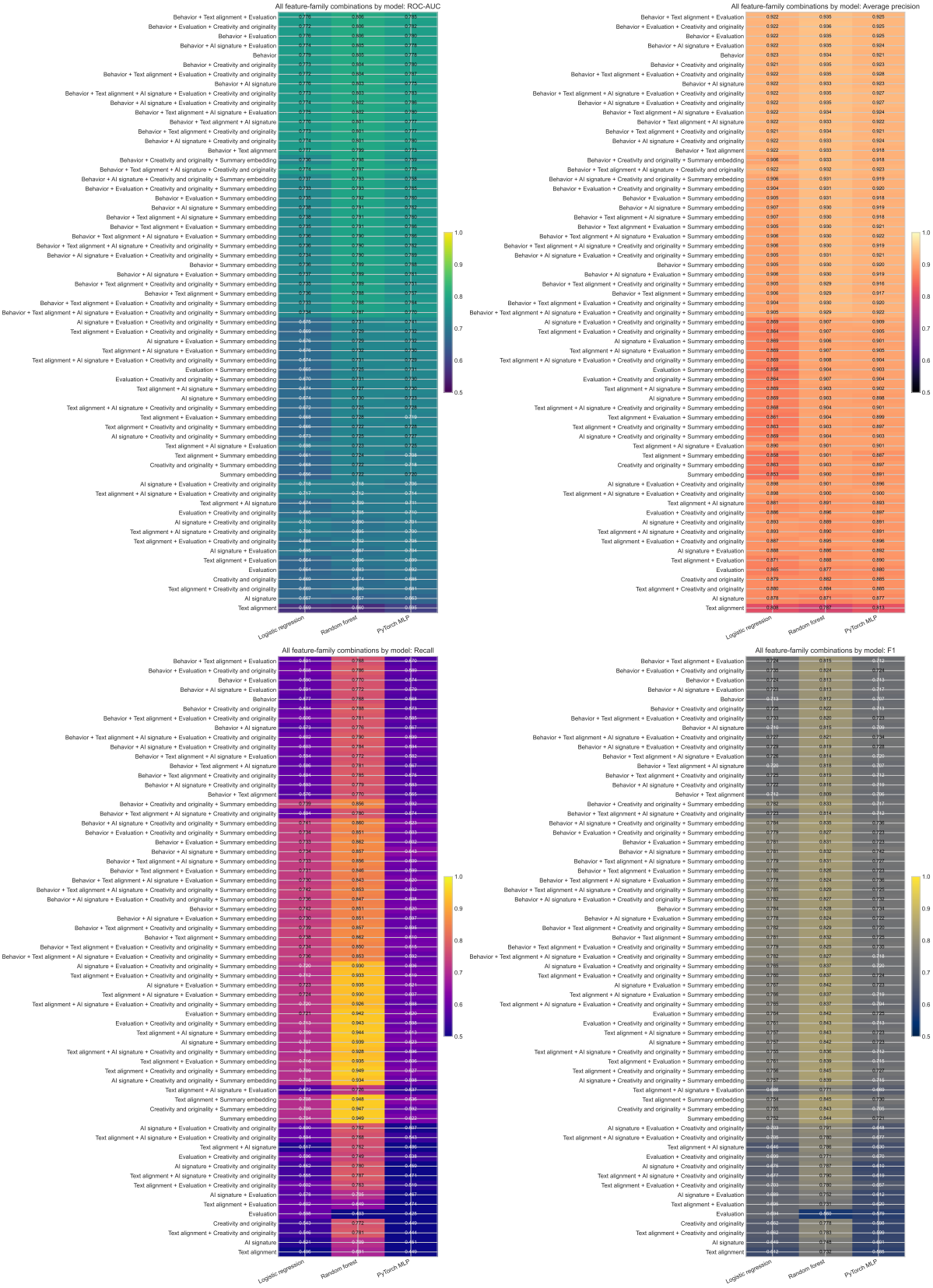


Figure A.20: Any AI assistance versus baseline control: classifier performance across feature-family combinations.

Notes: The figure reports out-of-sample classifier performance for distinguishing any AI-assisted human response (including those receiving low quality AI) from baseline human control responses. The analysis sample contains $N = 1647$ observations ($n = 1235$ Any-AI, $n = 412$ control). Columns correspond to logistic regression, random forest, and PyTorch MLP models. Rows correspond to all possible combinations of the feature families *Behavior*, *Text alignment*, *AI signature*, *Evaluation*, *Creativity and originality*, and *Summary embedding*. The four panels report ROC-AUC, average precision, recall, and F1. Rows are ordered by the highest ROC-AUC achieved by any model for that feature-family combination. The *Summary embedding* family is based on a 384-dimensional sentence-transformer representation of the written summary (`all-MiniLM-L6-v2`). In this broader comparison, the best-performing bundle combines *Behavior*, *Text alignment*, and *Evaluation*, while adding *Summary embedding* does not improve the top-performing specification.

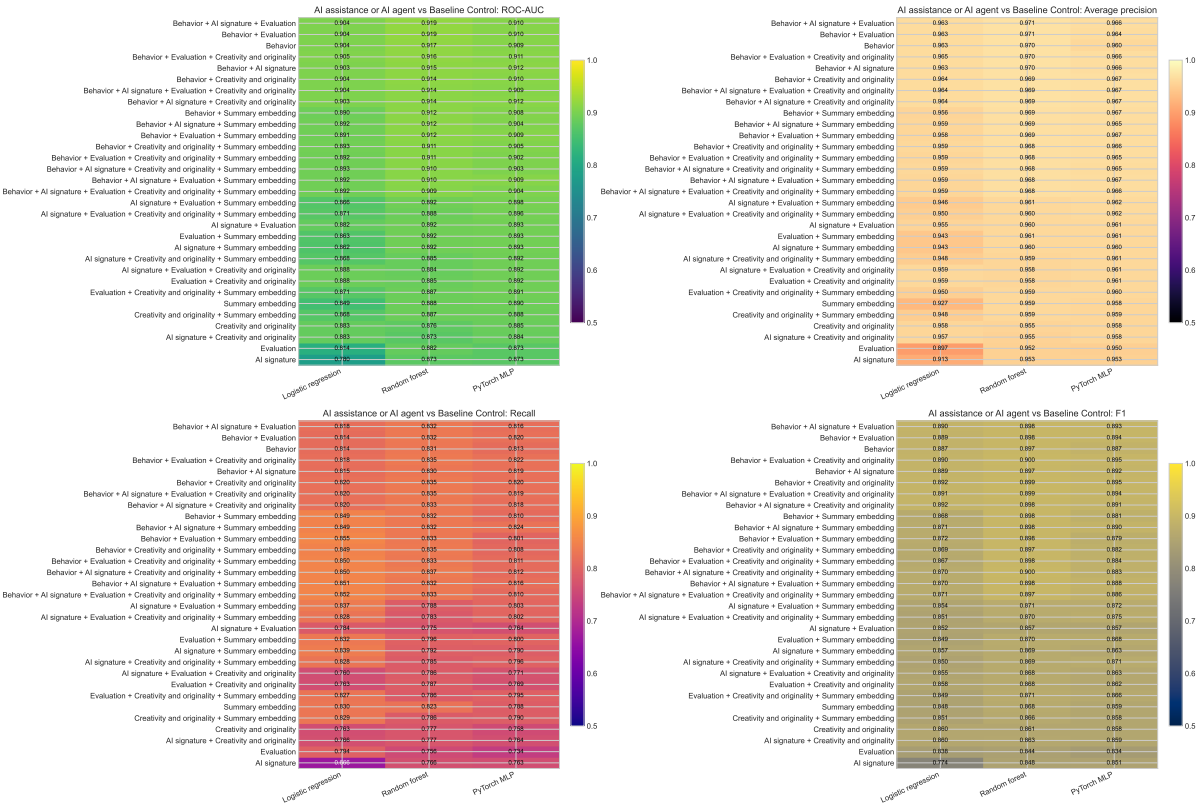


Figure A.21: Classifier performance across feature-family combinations: AI assistance or AI agent vs. Baseline Control

Notes: The positive class combines human `AI assistance` responses and all AI agent responses; the comparison group is Baseline Control. The four panels report cross-validated ROC-AUC, average precision, recall, and F1. Rows show feature-family combinations, and columns show the classifier used (logistic regression, random forest, and PyTorch MLP). Feature families are *Behavior*, *AI signature*, *Evaluation*, *Creativity and originality*, and *Summary embedding*. The summary embedding family is based on 384-dimensional summary-text embeddings from `sentence-transformers/all-MiniLM-L6-v2`. Larger values indicate better classifier performance. Sample sizes are $N = 412$ for Baseline Control and $N = 959$ for the combined AI group.

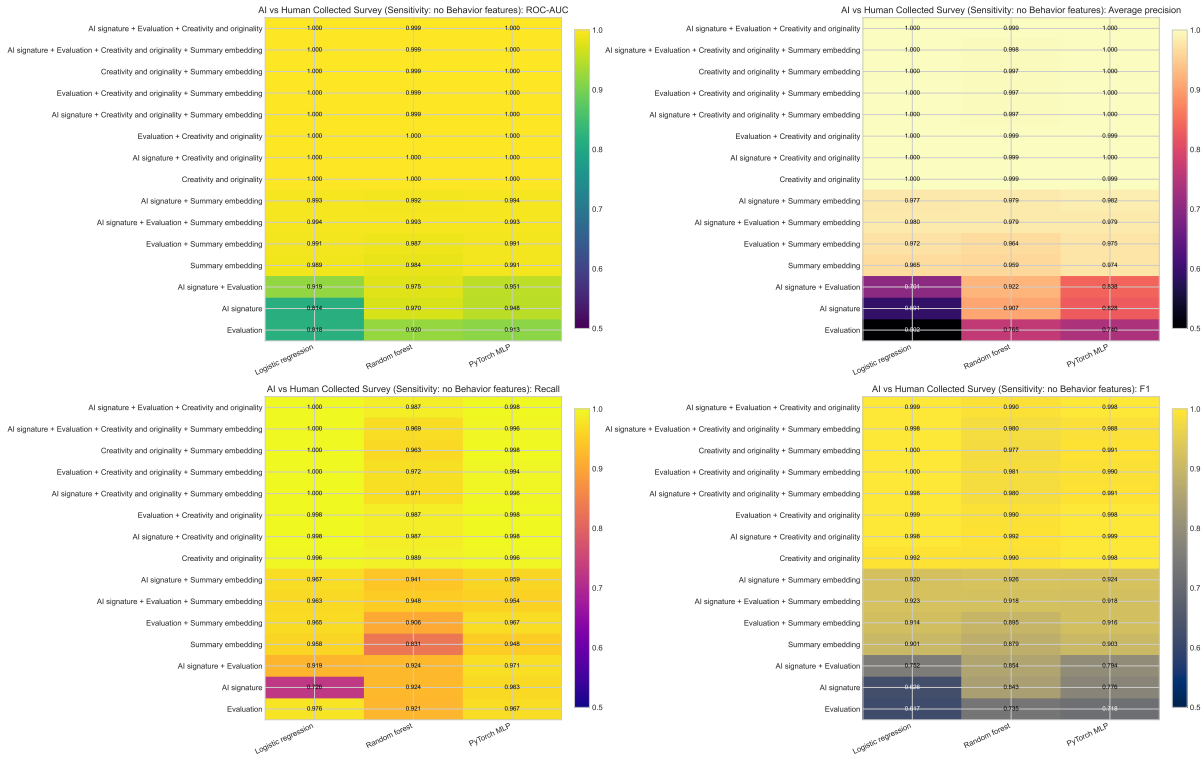


Figure A.22: AI agent versus human collected survey responses (sensitivity analysis excluding behavior features): classifier performance across feature-family combinations.

Notes: The figure reports out-of-sample classifier performance for distinguishing AI agent responses from the pooled human-collected survey sample after removing the full *Behavior* family. The human comparison group combines baseline control and all human AI-assistance arms. The analysis sample contains $N = 2190$ observations ($n = 543$ AI agent, $n = 1647$ human-collected survey). Columns correspond to logistic regression, random forest, and PyTorch MLP models. Rows correspond to all possible combinations of the remaining feature families: *AI signature*, *Evaluation*, *Creativity and originality*, and *Summary embedding*. The *Behavior* family is excluded entirely, so the models do not use timing, click, keystroke, or edit-telemetry variables. The four panels report ROC-AUC, average precision, recall, and F1. Rows are ordered by the highest ROC-AUC achieved by any model for that feature-family combination. The *Summary embedding* family is based on a 384-dimensional sentence-transformer representation of the written summary (`all-MiniLM-L6-v2`). In this sensitivity analysis, the first-ranked top-performing specification combines *Evaluation*, *Creativity and originality*, and *Summary embedding* with logistic regression, and multiple non-behavior feature bundles achieve near-perfect or perfect separation.